

Machine-Checked Dual-Write Recovery from a Commit Log

Andreas Andreakis

Abstract

Applications often need to make related facts durable in two independent systems without a transaction spanning both. If the process crashes after the second system accepts an operation but before a source-side checkpoint is written, recovery cannot tell from source state alone whether to retry. Retrying may duplicate the effect, while skipping may leave committed work incomplete. Transactional outboxes and change data capture move this dual write out of an application process, but relay delivery and checkpointing remain separate durable operations.

The engineering problem is familiar, and systems address it with retries, checkpoints, idempotency keys, and fencing, and call the result exactly-once delivery. Whether that guarantee holds depends on which event it counts, what evidence recovery requires, and how long that evidence must survive. Formal verification has covered adjacent problems, including transaction isolation, crash safety within one store, and protocols with a shared commit. The closest formal studies use model checking to verify particular outbox and log-delivery designs, and their results hold only for the designs and instances they check. Answering the three questions in general rather than for one design requires statements about arbitrary recovery policies, which finite enumeration cannot reach. We prove them in Isabelle/HOL, and to our knowledge they have not been machine-checked before.

The main result is an impossibility theorem for source-only recovery. We construct two reachable post-crash states with the same durable source-side state and different sink acceptance records. Any recovery policy based only on the source side must duplicate an effect in one state or leave it undelivered in the other. The same result holds for a deterministic deliver-then-checkpoint protocol whose only non-determinism is crash timing. When the sink exposes an authoritative, complete, and current acceptance record, recovery can compute exactly which committed operations remain undelivered, provided source coordinates distinguish operations. We extend the model to requests that remain in flight and to concurrent recoverers, and prove an arrival fence and a claim fence for these hazards. Finally, we show how bounded deduplication state and truncated source history limit the lifetime of the guarantee. A recovery design must therefore identify the sink's acceptance event, prevent stale actors from changing the result after it is read, and retain the required evidence for the full recovery horizon.

Keywords: dual writes, exactly-once delivery, crash recovery, change data capture, formal verification, Isabelle/HOL

1 Introduction

Consider an order service that commits shipped to its database. A relay sends a confirmation email for each committed order and records a checkpoint after every send. The process then crashes and restarts. Its checkpoint is now one order behind the commit log, so recovery sends the last confirmation again and the customer receives two copies. An inspection finds no error in the database as the rows are correct, the checkpoint is valid, and the store-level audit passes. This paper explains how a process that checkpoints after every send can still fail to provide exactly-once delivery.

The failure occurs because delivery and checkpointing are separate durable operations. The crash may occur after the mail provider accepts the request but before the process records the checkpoint. At restart there are two possible states. In one state the provider accepted the request, and in the other it did not. The service can read the same database, write-ahead log, outbox, acknowledgement log, restart counter, and checkpoint in both states. Sending the message again creates a duplicate in the first state. Skipping it leaves committed work incomplete in the second. The local checkpoint is not incorrect. It simply does not record whether the receiving side accepted the effect. Only a durable record at the receiving side can distinguish the two states.

Dual writes. The incident is an example of a *dual write*. Related facts become durable under independent authorities without one commit spanning both. The source may be a database, while the second authority may be a mail provider, broker, cache, search index, or another service. The model distinguishes a durable source commit from a durable sink acceptance. Section 2 and the theorem statements give the remaining requirements.

The usual remedy is to derive the second write from the source's committed log through change data capture (CDC) or a transactional outbox [1, 2, 3]. This approach removes the dual write from application code. Section 9 proves what it provides at the store tier. A derived copy is crash-safe exactly when it matches the committed log up to the log position it advertises. The relay must still deliver each derived event and record its own progress. These are again two durable operations without a shared commit. The log-based approach therefore moves the recovery problem to the relay. The replayable source log lets the relay retry failed deliveries and catch up after an outage. Recovery still depends on the acceptance evidence and controls provided by the sink.

Why formalize now. The engineering guidance for this problem is established. Prior work recommends deriving state from a log, retrying failed deliveries, using stable identities, and fencing stale workers [1, 2, 3, 4]. Systems that combine these mechanisms call the result exactly-once delivery. Yet such claims often leave unclear what counts as delivery, how recovery can determine whether it occurred, and how long the required evidence remains valid. This can make an information limit look like a checkpoint bug. It can also make a conditional guarantee look impossible when the required sink-side evidence and controls are available.

Formal verification has addressed nearby problems such as transaction isolation [5], single-store crash recovery [6, 7], and distributed protocols with a shared commit [8]. Those models do not decide whether an independent sink already accepted an effect. The closest formal studies use bounded model checking for outbox and log-delivery designs [9, 10]. To our knowledge, this is the first machine-checked proof of this recovery limit for all policies that use only the modeled source-side state.

Results. The results are organized around three parts of the recovery problem.

Source-side impossibility result. Sections 3 and 4 construct reachable post-crash states that agree on the durable information available to recovery and differ in the sink’s accepted record. Every policy that reads only the source side must choose the same recovery batch in both states. That batch duplicates a delivery in one state or leaves one undelivered in the other. Section 4 strengthens the result to one deterministic deliver-then-checkpoint protocol. The protocol maintains its cursor correctly, and crash timing alone creates the two indistinguishable local states.

Stale recovery information. Section 5 proves that recovery can compute the missing batch from an authoritative, complete, and current sink acceptance record under stated conditions. Sections 6 and 7 then identify two ways in which that read can become stale. A request from the crashed process may still be in flight, or another recoverer may act on the same crash. We prove an arrival fence for the first hazard and a claim fence for the second. Both mechanisms act at the sink’s acceptance boundary. The arrival fence also has a proved cost because it may reject an old request that would otherwise have completed later work.

Evidence lifetime. Section 8 proves the standard relationship between at-least-once delivery and a permanently deduplicated sink view. It then shows what changes when deduplication state expires or the source history is truncated. Once the sink forgets an accepted identity, a valid replay may be processed as new. Once recovery loses the content of the source history, it may no longer be able to distinguish work that requires delivery from work that was never committed. The guarantee therefore lasts only as long as the evidence required by the recovery procedure.

Scope. This paper presents a theoretical study of dual-write recovery. All theorems are machine-checked in Isabelle/HOL. The models, theorem statements, assumptions, and proof ideas are included in the text, so the paper can be read without the formal artifact. The negative results are constructions over specific machines and policy classes. They show that the stated failure is reachable and that every policy in the class fails on the construction, without claiming that every deployed system fails. The positive results hold under conditions stated with each theorem. We do not verify a broker, connector, mail provider, or deployed recovery implementation.

The model covers one source authority, one accepting endpoint, and one delivery direction, alongside in-flight delivery and concurrent recovery on separate machines. Section 10 lists further boundaries. The theory aims to identify the evidence and acceptance controls required by a recovery claim, not to certify a product.

Organization and reading paths. The paper provides two complementary reading tracks. Readers focused on formal foundations can follow the *Theory Track* (Sections 2-11), which develops the transition systems, impossibility results, fence constructions, and retention limits. Readers focused on practical systems can take the *Practitioner Track*, jumping directly to Section 12. That self-contained guide requires no proof mechanics and translates the results into operational guidance for direct dual writes and CDC outbox relays, complete with sink archetypes, hazard analyses, and recovery tests. Section 13 concludes.

2 The Model

All results use a common state model. This section defines the shared state, the first transition system, and the terminology used throughout the paper. Later sections extend the state with a checkpoint, an in-flight channel, concurrent recovery, retention, and the store tier.

Histories, frontiers, materialization. A *source event* e on a key k inserts or updates that key to a value, or deletes it. A committed *source history* H is a finite sequence of (coordinate, event) pairs whose coordinates are non-decreasing. A *frontier* f is a coordinate of that history, never a wall-clock time, a broker offset, or a sink-local sequence number. Given a base map b , the *materialization* $\text{Src}(b, H, f)$ gives each key the value of its latest event at a coordinate at most f (a delete yields absence), falling back to b where the key has no such event. Formally, histories are arbitrary lists of this shape and frontiers arbitrary coordinates. The non-decreasing well-formedness just described, and membership in a machine’s reachable executions, are imposed by stated assumptions and by the machines’ own transition rules, not by the raw data type. The *core execution state*

$$\sigma = (b, H, D, Q, \text{pend}, K, \text{fin}, \text{st}, \text{Ack})$$

carries the base, the committed history, an independently delivered downstream history D , an enqueued history Q with the pending set $pend$ of enqueued-but-undelivered events, the key scope K , the interval end fin , a run status $st \in \{\text{Running, Crashed } c, \text{Recovered}\}$, and a source-side acknowledgement history Ack . Labelled steps drive the state: commit to H , enqueue, deliver to D , acknowledge into Ack , crash (which freezes the run at a frontier), and recover. A completed recovery cycle returns the run to Running with a new generation. A two-event instance grounds the notation and returns throughout the paper: with $H = [(c_1, \text{ins } k \ 7), (c_2, \text{upd } k \ 9)]$ and an empty base, $\text{Src}(b, H, c_1)(k) = 7$, $\text{Src}(b, H, c_2)(k) = 9$, and the scoped obligations at c_2 are both events, in order.

Both stores are read through the one materialization operator at the same source frontier: $\text{Src}(b, H, f)$ is what the committed log says, $\text{Src}(b, D, f)$ is what the downstream holds.

Definition 2.1 (Scoped mismatch). At frontier f , the downstream *mismatches* the committed log on key k when $k \in K$ and $\text{Src}(b, D, f)(k) \neq \text{Src}(b, H, f)(k)$. Write $\text{clean}(\sigma, f)$ when no scoped mismatch holds at f .

When a crash can freeze such a mismatch where an observer sees it is a question about the *store tier*, and it has an exact answer, proved in Section 9. The subject of this paper sits above that tier, at a boundary the store never records.

The acceptance boundary. The *effect machine* extends σ with one field and one register. A state is $t = \langle \sigma, E, \gamma \rangle$: the core, an *emissions ledger* E , and a *generation* γ . The ledger records received effects such as emails sent, messages published, and webhook calls fired. It is append-only by construction: a received effect cannot be unreceived, so no step of any machine in this paper removes a ledger entry. No store field can read the ledger. It models what happened, not what the crashed side knows. The generation γ is the restart counter of Section 1. It identifies successive process runs, starting at zero. Each completed resume enters the successor generation. Related counters in deployed systems are called epochs or terms.

An *emission* is a triple $x = \langle n, g, (c, e) \rangle$: the *stamp* n (the length of the committed history when it fired), the generation g current when it fired, and the *payload* $\text{pay}(x) = (c, e)$. An emission is *justified* when the durable committed prefix covered it at fire time, a check the stamp makes readable from the state:

$$\begin{aligned} \text{justified}(H, x) &\longleftrightarrow \text{stamp}(x) \leq |H| \\ &\wedge \text{pay}(x) \in \text{take}(\text{stamp}(x), H). \end{aligned}$$

For a ledger L , $\text{pay}(L)$ is the ordered pointwise projection $[\text{pay}(x) \mid x \leftarrow L]$. On this first machine, delivery is instantaneous: an ordinary downstream delivery advances D and appends the received effect to E in a single transition.

Section 6 splits that collapsed send/receive step, because a whole class of hazards arises in the gap between the two.

Hazards and missing deliveries. Three predicates over t classify how a ledger can be wrong, and one checks for missing deliveries. The *obligations* at a frontier are the scoped, frontier-bounded committed events

$$\text{replay}(H, K, f) = [(c, e) \leftarrow H \mid c \leq f \wedge \text{key}(e) \in K],$$

and the hazard vocabulary is

$$\begin{aligned} \text{premature}(t) &\longleftrightarrow \exists x \in E. \neg \text{justified}(H, x), \\ \text{duplicate}(t) &\longleftrightarrow \text{some payload repeats on } E, \\ \text{unsafe}(t) &\longleftrightarrow \text{premature}(t) \vee \text{duplicate}(t), \\ \text{lost}(f, t) &\longleftrightarrow \exists p \in \text{replay}(H, K, f). p \notin \text{pay}(E), \\ \text{alo}(f, t) &\longleftrightarrow \text{replay}(H, K, f) \subseteq \text{pay}(E). \end{aligned}$$

unsafe is a safety verdict over what *was* emitted: something fired too early, or fired twice. It does not constrain what never fired. *lost* and *alo* are each other's complements and form the completeness axis. Read at a state, they say whether a frontier still has an undelivered payload. They make no liveness claim. They classify the state where a result is judged, not what some later execution might still deliver. The theorems apply them at post-recovery states, where the recovery being judged has finished its work. The verdicts are order-invariant. What was delivered matters, but its order does not. Multiplicity matters to duplicate by definition, and to the completeness pair only under the ascending-coordinate condition P4 (Section 5). "Exactly-once at f " always means the conjunction (no hazard, and at-least-once at f), and the bare phrase is never used as a formal name in this paper: every use is frontier-relative and counted as its theorem states.

The core's acknowledgement history Ack records *source-side* acknowledgements, and it supports no delivery conclusion: equal acknowledgement histories can coexist with different emission ledgers. An acknowledgement is what the caller heard, not what the other side did. The opening incident turned on this distinction, and the model makes it structural.

What each side must provide. The model abstracts over both ends. On the source side, the analysis assumes only a durable, ordered, append-only committed history: each fact either is in H , with a stable coordinate, or is not. How a fact became durable lies outside the model: no relational store, multi-object transaction, or isolation discipline is assumed anywhere. A write-ahead log, an event store, a replicated queue, or a store limited to single-document writes each yields an H . Strictly ascending coordinates are required twice (condition P4, Section 5). They keep the sink-delta re-drive batch duplicate-free, and they lift the counting results from set level, by payload, to per instance. A source that can lose committed content is not left unmodeled: this section's machines never shrink H , and truncation gets its own regime

in Section 8. And a fact with no single committing write has no place in H until one write is designated as its durability event. A fact spread across independent writes with no such point reproduces this paper’s problem one level down, inside the source.

On the sink side, the judged event is *durable acceptance*, the moment the receiving authority durably admits the operation: a broker’s acknowledged append, a mail provider accepting a send request, a store committing a put. Nothing in this paper reaches past acceptance, not into internal processing, onward delivery, or what a user interface eventually shows. Sinks differ in what they let recovery *read*, not in their kind. A sink exposing its own durable, complete per-operation accepted record supplies the raw material for the positive results ahead. Their conditions are stated where they are proved, starting in Section 5. A sink readable only as current contents gets strictly less, and what that read can and cannot separate is itself proved (Sections 3 and 4). The observation bounds in Sections 3-4 apply when recovery can read only the modeled source-side state. Source acknowledgements do not distinguish the constructed cases. The obligation runs one way, from the source’s record to the sink’s acceptance. Obligations running both ways are reached here only one direction at a time, and if even the source keeps no such record, the question cannot be posed at all.

Re-drives, policies, observations. A *re-drive policy* P maps a state to a batch of payloads. The re-drive relation $t \rightsquigarrow_{P(f)} t'$ is one atomic recovery step doing two things: it heals the store (the delivered history is reconciled to the committed image at f , so $\text{clean}(t', f)$ holds after *every* re-drive, whatever the policy), and it appends $P(t)$ ’s emissions to the ledger, stamped with the current history length and generation. That the step is one atom is a deliberate model boundary. Section 10 discusses this atomicity assumption, and Section 5 shows that the central bound does not lean on it. An *observation* is any function obs of the state, of any result type. A policy P *factors through* obs when $P = g \circ \text{obs}$ for some g . Factoring is the paper’s only notion of what recovery reads: a policy that factors through obs can use obs ’s output and nothing else. We consider three observations. The complete modeled store tier includes both source and downstream histories, with $\text{obs}(t) = \text{core}(t)$. The second observation adds the generation, $\text{obs}(t) = (\text{core}(t), \text{gen}(t))$. The third also includes the sink’s acceptance record. Section 3 rules out policies based on the first two observations. Section 4 then studies a fixed protocol using the durable-local view, which excludes downstream content as well as the acceptance record. Section 5 gives a correct sink-reading policy under its stated conditions.

Mapping the model to systems. Table 1 fixes the substitution between practitioner vocabulary and model objects. Two rows distinguish roles. An outbox table is the committed log when a relay reads it as its source, and it is an

Table 1. What the model’s objects stand for. The mapping is one-way (theorems name only the left column), and the outbox rows are role-split on purpose.

Model object	What it stands for
emission x on the ledger E	the second write: an email, a webhook call, a cache fill, an index update
accepted record A (§6)	the sink’s durable record of what it has accepted
wire W (§6)	retries, socket buffers, requests still in flight
store state inside σ	source/downstream histories, queue and pending state, run status, source acknowledgements
cursor (§4)	a durable checkpoint of delivery progress
generation γ (§2), fence φ (§6)	producer generation and the sink’s acceptance threshold
committed log H	the source’s log, <i>or an outbox table read as the relay’s source</i>
accepted record A	sink-owned acceptance state colocated with an outbox only when updated atomically with acceptance (§9)
journal J , floor fl (§7, §8)	the upstream log as justification authority, retention and compaction

accepted record only if it also holds sink-owned acceptance state updated atomically with acceptance. Merely consulting a source outbox or a relay cursor does not turn either into the sink’s record. Which theorems govern a physical table depends on the role its fields actually play. The substitution is one-way, as theorems are stated over abstract model objects rather than concrete technologies like webhooks or caches. The target-agnostic objects subsume them. We avoid “consistent” because it conflates several properties. When this paper means the stores agree at a frontier it says mismatch-free, and where it means delivery went right it names the hazard and the frontier.

Scope of the results. Negative results concern the policy classes and schedule families named in their statements. Positive results hold under their stated conditions. The store-tier equivalence in Section 9 ranges over all implementations satisfying its assumptions.

3 The Store-Side Observation Bound

After a crash, recovery must decide what to re-send. The first two theorems show that no policy computed only from the durable store and its control plane is correct on all reachable states. The state available to recovery does not reveal whether the sink accepted the operation.

The designed pair. We construct two reachable post-crash states with the same store state and generation but different acceptance records. The policy being tested chooses

what to resend from these states. It need not have produced the histories that led to them. Section 4 shows how crash timing creates this ambiguity in a fixed protocol.

Two runs of the effect machine commit the same two facts and crash with the second delivery enqueued but un-sent. Their first recoveries then diverge in one respect: the machine's recovery rule takes a *reconcile index* m marking where re-delivery starts, and the two runs use different values of it. With $m = 1$, reconcile emits the missing payload, and the emissions ledger records it. With $m = 2$, reconcile rebuilds the store and emits nothing, an advanced-cursor pathology one notch away from the incident's stale-cursor duplicate. Both runs then resume and stand crashed again at the same frontier c_2 with the store fully caught up. At that point their formal cores are byte-equal: base, committed and delivered histories, queue and pending state, scope, interval, status, and source acknowledgements all agree, and so do their generations. The earlier m choices were parameters of actions already taken. No trace of them remains in either state. Both states are hazard-free, and both have genuinely emitted work ($E \neq []$). One member still has an undelivered payload, and the other has already delivered it. They differ only in the emissions ledger:

$$\text{pay}(E(t_1)) = [(c_1, e_1), (c_2, e_2)], \quad \text{pay}(E(t_2)) = [(c_1, e_1)].$$

Theorem 1 (Observation Bound). *On the effect machine there are reachable states t_1, t_2 and a frontier c_2 with*

$$\text{core}(t_1) = \text{core}(t_2), \quad \text{gen}(t_1) = \text{gen}(t_2), \quad E(t_1) \neq E(t_2),$$

both hazard-free and with nonempty emission ledgers, such that for every observation obs of any result type with $\text{obs}(t_1) = \text{obs}(t_2)$, and every policy $P = g \circ \text{obs}$, there are re-drives

$$t_1 \rightsquigarrow_{P(c_2)} t'_1 \quad \text{and} \quad t_2 \rightsquigarrow_{P(c_2)} t'_2$$

with $\text{clean}(t'_1, c_2)$ and $\text{clean}(t'_2, c_2)$, and

$$\text{unsafe}(t'_1) \vee \text{lost}(c_2, t'_2).$$

Any policy that observes the same state in both executions must choose the same batch. If the batch contains the missing payload, one execution receives a duplicate. If the batch omits it, the other execution remains incomplete. Adding uncommitted work only creates a premature effect. Both repaired stores are mismatch-free, so a store audit cannot distinguish the failure. Figure 1 shows this argument.

The observation is otherwise unrestricted. It may be any function of the state, with any result type, and only equality on the constructed pair is assumed. The result applies only to information represented by the modeled observation.

Theorem 2 (Control-Plane Bound). *Every store-measured policy, computed as $P(t) = g(\text{core}(t))$ for every t , and every store-and-generation-measured policy, computed as $P(t) = g(\text{core}(t), \text{gen}(t))$, is defeated on a reachable effect-machine pair as in Theorem 1: both re-drives leave the store mismatch-free at the pair's frontier, and the outcome is a hazard on one member or a loss on the other.*

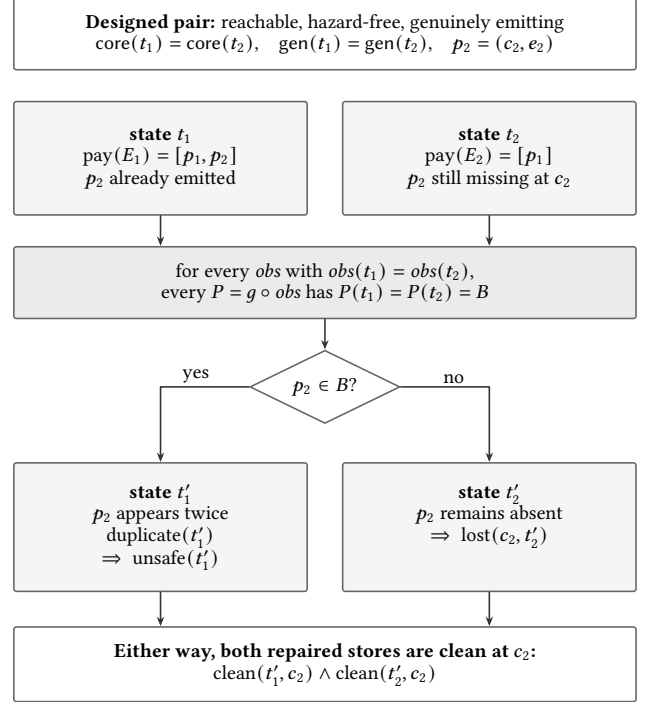


Figure 1. The constructed pair for Theorems 1-2. The states have equal core and generation but different emissions ledgers. Any common observation yields the same batch, which duplicates p_2 in t_1 or leaves it undelivered in t_2 . Both repaired stores are mismatch-free.

Proof idea. Policies of both classes agree on the designed pair, because the pair is equal on core and generation. Theorem 1 then applies. The mechanized proof checks the three-way split directly (empty batch, batch containing a payload from the committed prefix, non-empty batch entirely outside it), and each case yields one of the stated outcomes. $\square$

General and pair-specific outcomes. The disjunct in Theorem 1 is $\text{unsafe} \vee \text{lost}$, and since unsafe itself splits, the general result has three outcomes: duplicate, lose, or fire prematurely. At the designed pair the checked statement sharpens to the practitioner's two, because in the premature case the skip-side member still lacks its delivery: the verdict there is *duplicate a delivery or leave it incomplete at the judged frontier*. Both forms are machine-checked. The three-outcome statements remain the general results, and the two-outcome readings apply to the designed pairs used in the examples. The loss outcome carries its standing disclosure: it is judged where the recovery under examination has finished, not a claim that no later recovery could still act.

Store and ledger health are independent. Store health and ledger health are orthogonal on this machine, in both directions: states with a broken store and a clean ledger are reachable, and so are states with a healed store and a

broken ledger. A second crash-and-re-drive cycle can raise the same payload's count in the emissions ledger to three. An accurate delivered-count cursor persisted *outside* the store would separate the pair, by being a sink-side record under another name, which is precisely the observation class Section 5 builds on. Distinguishing the pair alone does not establish a recovery guarantee. The positive results below apply to a specific policy under stated conditions.

There is also a structural version of that boundary. A proved dichotomy partitions all policy functions: every re-drive policy either is a function of the modeled store and generation (and fails on the pair in Theorem 2) or distinguishes some two states that agree there and differ on the ledger. The witnessing states of the second class need not be reachable. The split is extensional, a statement about information rather than operations. Its value is exhaustiveness: at this abstraction there is no third kind of policy.

Scope of the observation bound. Both slogans that circulate about this problem fail against these results. “Exactly-once is impossible” is too coarse: Sections 5, 6, and 8 prove exactly-once results at their frontiers, under conditions a system can satisfy. The true statement is the bound above (no policy blind to the sink’s record achieves it), together with the conditions under which the positive results ahead hold. “Just read the sink before re-driving” fails in the other direction: once deliveries travel, Section 6’s straggler defeats every channel-blind batch selector on the constructed pair under the unfenced re-drive relation. The positive result changes acceptance instead of reading more.

4 The Checkpoint Window

The observation bound uses two earlier reconciliation choices. To show the same ambiguity in a fixed implementation, we add a durable cursor and study a deterministic deliver-then-checkpoint protocol. A crash before delivery and a crash after delivery but before checkpointing leave recovery with the same durable-local view and different sink records. The cursor is part of the common view.

The checkpointed machine. Extend the effect machine with a single durable field: a progress cursor. Delivery and checkpointing are separate transitions: an ordinary delivery advances the downstream and the ledger as before, and a separate *persist* step advances the cursor by one, permitted only when there is an actual delivery to acknowledge. A crash may land between them, and that window is the subject of this section. The machine’s one recovery rule is the effect machine’s emitting reconcile with its index m instantiated by the machine’s own cursor: recovery re-fires the committed suffix from wherever the durable cursor points. The free parameter of Section 3 is thereby instantiated by a durable

One deterministic protocol. The adversary only places the crash

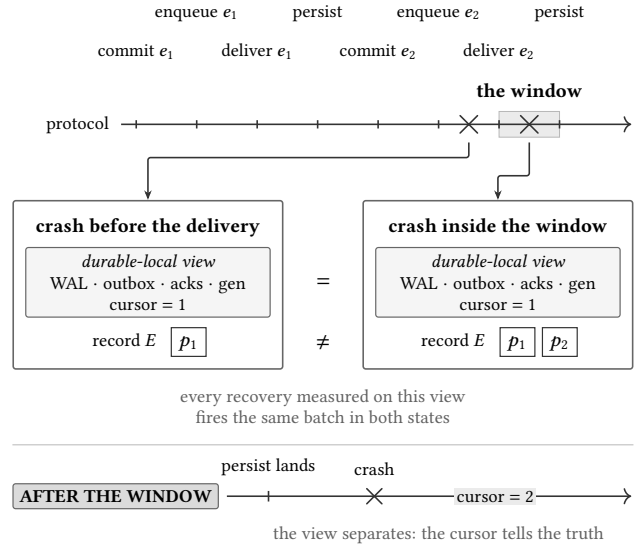


Figure 2. The checkpoint window. Crashes immediately before the second delivery and between that delivery and its persist produce equal durable-local views but different emissions ledgers (Theorem 3). A crash after persist advances the cursor and separates the states.

cursor: this machine’s recovery action carries only the frontier, and nothing else is left for an adversary to choose at recovery time.

Fixed protocol and crash schedule. On this machine we pin one *protocol*: a function from states to next actions implementing the faithful deliver-then-persist discipline over the running two-event workload (commit e_1 , enqueue e_1 , deliver e_1 , and persist, then repeat these four actions for e_2 and stop). The protocol itself never crashes. A run consumes a *schedule* over a two-letter alphabet: Go executes the protocol’s unique next action, Crash is the adversary’s only move, the machine’s ordinary status-flip crash, at the point in the run where it lands. The endpoint of a run is a function of its schedule: crash placement is the only nondeterminism anywhere in the construction, and no per-run parameter exists that could distinguish runs.

Two schedules matter: Go⁷ Crash and Go⁶ Crash (the crash landing just after the second delivery, versus just before it). Both runs end crashed with cursor 1: the first inside the second deliver-to-persist window, the second right before that window opens. At the first run’s endpoint, the emissions ledger holds both payloads. At the second it holds only the first, exactly the ledgers of Section 3’s pair, this time produced by one fixed protocol (Figure 2).

What recovery can read. Call the *durable-local view* of a state everything a recovery procedure of the crashed process reads without querying the sink side: the base map, the

committed source log, the enqueue (outbox) log, the acknowledgement log, the scope, the interval end, the run status, the recovery generation, and the cursor. Write $\text{dview}(\kappa)$ for that projection, and call a policy *durable-local-measured* when it factors through it. Three parts of the full state sit outside the view, each for a stated reason. The emissions ledger is the sink-side record. Excluding it is the entire question. The downstream store's delivered content is excluded because reading it is reading the sink: it is not local state, and the pair below is provably not equal on it, so that read genuinely separates the members. The dichotomy at this section's end places such readers outside the defeated policy class. The pending delivery window is volatile by this machine's own modeling assumption: it is the crashed process's in-memory buffer, which recovery in this model does not retain. For each exclusion, a companion result shows that adding the excluded component back changes the verdict. The crash transition keeps every field. The model restricts which of those fields recovery may read after the crash.

Theorem 3 (Checkpoint Dilemma). *On the checkpointed machine there are reachable states κ_1, κ_2 , the endpoints of the two protocol schedules above, with*

$$\text{dview}(\kappa_1) = \text{dview}(\kappa_2), \quad E(\kappa_1) \neq E(\kappa_2),$$

both hazard-free and with nonempty emission ledgers, such that every durable-local-measured policy P is defeated at the finish frontier c_2 : there are re-drives $\kappa_1 \rightsquigarrow_{P(c_2)} \kappa'_1$ and $\kappa_2 \rightsquigarrow_{P(c_2)} \kappa'_2$, both leaving the store mismatch-free at c_2 , with

$$\text{unsafe}(\kappa'_1) \vee \text{lost}(c_2, \kappa'_2).$$

At this pair the checked verdict again sharpens to two outcomes: duplicate a delivery, or leave one undelivered at the judged frontier.

The cursor is inside the view: recovery reads it, and it is equal on the two members. So are the committed log, the outbox, the acknowledgements, the status, and the generation. The pair does not arise from pathological bookkeeping. It is the deliver-then-persist window, crossed by a crash, seen from the inside.

The deterministic protocol. The defeat quantifies over all durable-local-measured policies, and the machine's own faithful recovery inhabits the class, so the theorem applies to it. A checked trace also runs it concretely. Fired from the two view-indistinguishable members, the same parameter-free recovery transition is exactly right on the lagging member (store healed, no hazard, no delivery missing at c_2) and produces an effect-unsafe duplicate on the delivered member, every entry of which is individually justified. That is the familiar at-least-once window of checkpointed delivery, stated and checked as a theorem. Recovery emits p_2 after either crash, but only the crash after delivery makes that emission a duplicate.

Relation to the equal-core pair. One distinction separates this result from Theorem 1. That pair was equal on the whole modeled core. This pair is equal on the durable-local view and provably *not* equal-core: the delivered member's downstream history and pending window record the delivery, and the model exposes this difference. The two shapes are complementary. The mechanization also proves that before the first resume, no reachable pair of crashed states has equal cores and ledgers that differ in their delivered payloads, so an equal-core, payload-divergent pair generated by one crash-timed protocol at generation zero cannot exist. Theorem 1 is the wider information bound over every observation that agrees on its equal-core pair. Theorem 3 is the protocol-specific bound over the reads a crashed implementation actually has. The two theorems therefore cover different recovery observations.

On this pair, reading the *downstream store's content* separates the members because it contains the delivered copy, so a content reader distinguishes this pair. That read is not store-local information. It is a read of the sink, in its cheapest form. But it does not survive Section 3: on the equal-core pair the downstream contents agree, so those reads do not distinguish the two states. Only the sink's per-operation accepted record distinguishes *both* regimes: the acceptance history, not the current contents. Reading the sink is therefore two different acts, reading its state and reading its acceptance history, and the distinction returns as a theorem boundary in Sections 8 and 9.

Policy classes and the cursor. As in Section 3, the split is exhaustive: every policy on this machine either factors through the durable-local view (and the pair defeats it) or distinguishes some two states agreeing on that entire view, which means it reads the downstream content, the volatile window, or the ledger. The cursor itself is not the defect: if the crash lands *after* the final persist, the cursor reads 2, which by itself separates that state's view from both pair members. Outside the deliver-to-persist window the cursor is accurate. The dilemma is the window, not the cursor's bookkeeping. That is why no amount of care in maintaining the checkpoint, and no smarter encoding of it, closes a gap that sits between two durable writes.

5 Reading the Sink

The pairs in the last two sections differ in the sink-side accepted record. A policy that reads this record can compute what is still missing. The *sink delta* at frontier f is the committed obligations not yet on the record:

$$\Delta(f)(t) = [p \leftarrow \text{replay}(H, K, f) \mid p \notin \text{pay}(E(t))].$$

Here $E(t)$ is the model's authoritative, complete record at the frontier of the decision. That assumption is necessary. A stale replica, a paginated status endpoint, or a lagging projection of the sink's state is a *different* observation, and

nothing below applies to it without a separate argument. The guarantee is exact, and it is conditional. The result requires four conditions:

- P1. *Hazard-free*: $\neg \text{unsafe}(t)$ (the record carries no duplicate and nothing premature).
- P2. *Crashed*: the run's status is Crashed c for some c .
- P3. *In-interval*: $f \leq \text{fin}$.
- P4. *Ascending*: the committed history's coordinates are strictly increasing.

Theorem 4 (Sink-Reading Escape). *On the effect machine, under P1-P4, the sink-delta re-drive at f exists, and every re-drive $t \rightsquigarrow_{\Delta(f)} t'$ leaves the store mismatch-free at f , hazard-free, and complete:*

$$\text{clean}(t', f), \quad \neg \text{unsafe}(t'), \quad \neg \text{lost}(f, t').$$

Proof idea. Existence follows from the atomic reconcile. The three conclusions follow from the delta's shape. It never re-emits a payload already on the record, so no duplicate occurs. It emits only committed obligations, stamped at the full history, so nothing premature occurs. It includes every payload still missing at f , so no loss remains. None of the three is automatic: each depends on the batch being computed from the same record the hazards are judged against. $\square$

On the checkpointed machine of Section 4, the same policy also separates that section's pair. The proof confirms that the delta is not durable-local-measured. It uses information outside the defeated class instead of refining a policy inside it.

Why ascending coordinates are required. P4 does double duty. It is what makes the delta's own batch internally duplicate-free, and it is what licenses the strongest reading of the conclusion: each committed obligation delivered *per instance* exactly once. Without it the count is set-level, by payload. That distinction matters. An equal-coordinate aliasing counterexample in the mechanization refutes the per-instance strengthening without ascending coordinates at the deduplication tier (Section 8). One further property comes free and one does not: the delta emits in committed source order (a proved fact), while ordering across a real transport is a different guarantee, which this paper does not provide. Section 6 models exactly where it is lost.

Normal execution. The delta chooses the right batch at recovery. A companion result proves the running discipline that keeps a live, emitting system hazard-free between recoveries, as three record-local guards:

- G1. *Fire-time justification*: an ordinary publish fires only a payload the committed history holds at fire time.
- G2. *Publish freshness*: it fires only a payload not already on the record.
- G3. *Fresh reconcile suffix*: a re-drive's suffix is internally distinct and wholly absent from the record.

Every run keeping the three guards stays hazard-free. The folklore version of this rule has two guards. A separate counterexample shows why each guard is required, and suffix freshness is provably necessary once a re-drive can meet a record that already holds part of its window. These guards do not guarantee completeness. An empty re-drive suffix satisfies them while silently skipping work. The guards constrain what fires, not what fails to fire.

Interrupted and repeated recovery. Under P1-P4, the small-step variant can deliver a prefix of the sink-delta batch and stop. Within this recovery window, the frontier and every field except the ledger remain fixed. Recomputing the delta returns the undelivered suffix. Partial rounds remain hazard-free. Delivering the remainder and repairing the store completes recovery.

A separate result covers repeated atomic sink-delta recoveries and Resume steps. Ordinary execution between them must satisfy the model's guards. From a hazard-free start, with ascending coordinates at each re-drive, every completed re-drive is exactly-once at its own frontier. Neither result ensures eventual completion. The status-only Recover step can mark a run Recovered without repairing its store. It is distinct from the sink-delta re-drive of Theorem 4.

The observation bound also holds in the small-step extension, including finite adaptive sequences of per-entry choices that cannot read the sink record. It therefore does not require atomic recovery. Crashes inside the fenced recovery operation of Section 6 remain outside that model.

Detecting omissions. Exactly-once has two conjuncts, and operators observe them asymmetrically. A duplicate is loud: the customer gets two emails, and an incident is opened. A frontier-relative omission is silent: a cursor advanced past an undelivered send leaves a record with no hazard at all, and a checked trace reaches a post-recovery state whose computed hazard verdict is clean while a committed operation remains undelivered at the judged frontier. The opening incident was the loud kind. The quiet kind passes the same store-level audit, which is why completeness needs to be checked separately.

6 Stragglers and the Arrival Fence

Theorem 4 was proved on a machine where a send and its acceptance coincide. Real deliveries travel. They wait in retry queues and socket buffers, arrive out of order, or vanish. The channel machine models that traffic, and on it the sink-delta policy of Section 5 stops being enough: a read cannot see what is still in flight.

The channel machine. A channel state is $u = \langle t, W, A, \varphi \rangle$: an inner effect state t , the wire W (emissions sent and not yet resolved), the sink's durable *accepted record* A , and a fence $\varphi \in \mathbb{N}$, zero until set. An ordinary publish adds its emission to W and to the inner ledger E , which here records sends

rather than acceptances. An arrival resolves one in-flight emission, in any order (the wire reorders freely), and the sink accepts it into A exactly when its generation clears the fence, $\varphi \leq \text{gen}(x)$, dropping it otherwise. An in-flight emission may instead be lost, as a first-class step. The bookkeeping is conservative (every sent emission is at all times accounted for as in flight, accepted, or dropped), and a crash moves the run's status and nothing else: the wire, the accepted record, and the fence all survive it, because a process crash does not empty network buffers. Hazards and completeness are now judged at the accepted record ($\text{unsafe}_A(u)$ for a premature or duplicate *accepted* entry, $\text{alo}_A(f, u)$ for at-least-once read there), and exactness at f is the conjunction

$$\begin{aligned} \text{eo}(f, u) &\longleftrightarrow \neg \text{unsafe}_A(u) \wedge \text{alo}_A(f, u), \\ \text{alo}_A(f, u) &\longleftrightarrow \text{replay}(H, K, f) \subseteq \text{pay}(A(u)). \end{aligned}$$

The unfenced policy re-drive studied below atomically repairs the store and appends its batch to both E and A . The batch is accepted synchronously without changing the wire or fence. Cursor-based recovery, using the suffix replay described in Section 4, instead queues its re-sends on the wire. Theorem 5 concerns the unfenced recovery rule just described. The later fenced rule is outside its scope because it changes the fence. Deployed systems keep a cousin of the generation under another name: Kafka's idempotent producer carries a broker-checked *epoch* [11], and this paper uses that word only for the cited mechanism.

An old request can arrive after recovery. Figure 3 shows an old send that remains in flight across the crash. Recovery reads the accepted record, sends the missing payload, and resumes in a new generation. The old request then arrives and the sink accepts it, producing a duplicate. Every entry is individually justified. The duplicate comes from the run's own valid send in the previous generation, not from malformed traffic.

When the old request arrives *before* the read, the read sees it and the delta is empty. This control shows that the sink read is correct when it occurs. It becomes stale only when the wire still contains work that can arrive after the read.

Definition 6.1 (Channel-blind policy). A policy P on the channel machine is *channel-blind* when it factors through everything but the wire: for some g ,

$$P(u) = g(\langle t, A(u), \varphi(u) \rangle) \quad \text{for all } u,$$

where t is the whole inner state, including the store, ledger, and generation.

Everything durable is inside the visible triple, the accepted record and the fence included. The one field outside it is the wire itself.

Theorem 5 (Wire Bound). *On the channel machine there is a reachable pair u_1, u_2 that agree on the visible triple. One has the crashed process's own send still in flight, while the other*

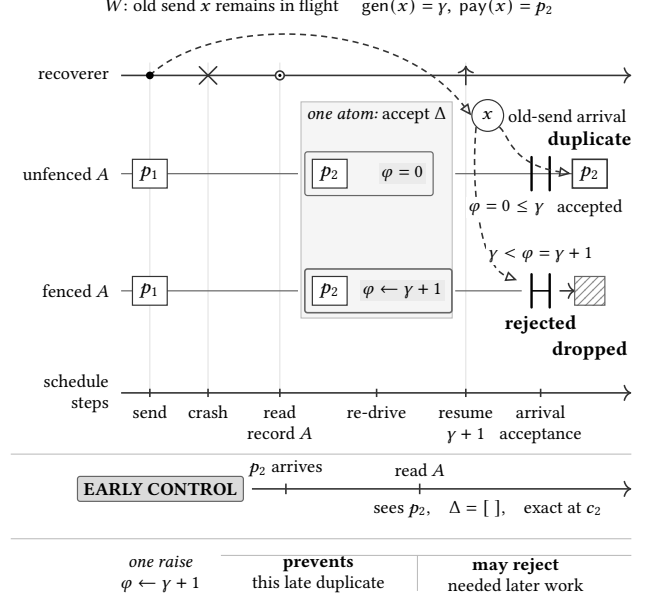


Figure 3. An old send crosses the sink read. Without a fence, its later arrival duplicates the accepted record. The fenced re-drive atomically lands the recovery delta and raises φ , so the arrival is rejected (Theorems 5-6). The lower rails show the benign early-arrival control and the rejection of later work by the fence (Corollary 6.1).

has an empty wire. This pair defeats every channel-blind policy P under the unfenced re-drive relation: there are re-drives of both members at c_2 whose outcomes satisfy

$$(\exists v. u'_1 \xrightarrow{\text{arrive}} v \wedge \text{unsafe}_A(v)) \vee \neg \text{alo}_A(c_2, u'_2).$$

Either the straggler's arrival leaves the first member's accepted record unsafe, or the second member still lacks a delivery at c_2 .

Because the policy cannot observe the wire, it chooses the same batch for both states. If the batch carries the missing payload, the in-flight member's straggler arrives at the never-set fence and the record duplicates. If it omits it, the empty-wire state has no pending request that could deliver the missing event and remains incomplete. The sink-delta policy of Section 5 is in the defeated class (Δ reads the accepted record and never the wire), and the late-arrival example above demonstrates this failure. A variant of the opening example has the same hazard: a send left in a retry queue at crash time can produce a duplicate after an otherwise correct sink read. The hazard is not an artifact of the model. The mechanization separates the late-arrival example's final record from everything the instant-delivery machine can reach, so the guarantee of Section 5 was not false on its own machine. That machine could not express this failure.

The arrival fence. The fenced re-drive at f is one atomic act: heal the store, compute Δ from the accepted record, land

the batch in the sent ledger and synchronously in A (a recovery's own re-sends never traverse the fenced wire), and set the fence to the successor of the crash-time generation, $\varphi := \gamma + 1$. From then on the sink rejects arrivals from generations below the fence. The guarantee has four conditions corresponding to P1-P4, read at the accepted record (record hazard-free, crashed, in-interval, and ascending coordinates, called Q1-Q4) and, for the continuation results below, the machine's reachability invariants.

Theorem 6 (Arrival Fence). *On the channel machine, under Q1-Q4, the fenced re-drive at f exists, and every fenced re-drive $u \rightsquigarrow u'$ leaves the store mismatch-free at f , is exactly-once at f on the accepted record, and sets the fence to the successor generation:*

$$\text{clean}(u', f), \quad \text{eo}(f, u'), \quad \varphi(u') = \text{gen}(u) + 1.$$

The theorem assumes strictly ascending coordinates because duplicate-freedom of the accepted record depends on them. The theorem certifies this admission mechanism under the machine's crash-time stamping discipline. It does not say a generation fence is the only possible way to neutralize a wire. A sink that atomically deduplicates on stable event identity at admission, for instance, is a different mechanism with different assumptions, unmodeled here.

Fence placement. The composite's shape is forced by a discipline the machine inherits: re-drives stamp at the crash-time generation (the reconcile is generation-constant, and resume bumps the counter only afterwards), so an earlier process run's in-flight emission and the recovery's own re-drive carry the *same* generation, and no fence value separates them on the wire. Set the fence at resume instead, and the straggler slips in during the gap. Setting it in a separate step before the re-drive would cause a recovery whose own sends traveled the wire to fence out its own work. Synchronous acceptance plus an atomic rise is the one sound point in this rule family. It is a design argument over the stamping discipline, while the two refuted reorderings that *are* theorems live one machine over, in Section 7. Figure 4 places the sound rise beside its failing neighbors.

The fence's policy is still the channel-blind Δ . Nothing reads the wire. That is the central point: more recovery-side reading does not flip the verdict (that was Theorem 2), while the same generation tag, consulted at *acceptance*, does. The fence works by changing what the sink admits, not by reading more.

Cost of the arrival fence. The following corollary shows that fencing can reject a delayed delivery needed at a later frontier.

Corollary 6.1 (Rejection of later work). *There is one schedule over fenced and unfenced twins that differ by one rule. Recovery occurs at frontier c_1 with a straggler for c_2 in flight. On this schedule, the unfenced run's straggler arrives and completes*

the record at c_2 , exactly-once there with the payload counted once, while the fenced run drops the same straggler: its result remains incomplete at c_2 , and exactness holds at the fence's own frontier c_1 .

The fence guarantees exactly-once acceptance at the chosen frontier. It can still leave later work undelivered by rejecting the old request that would have delivered it.

Corollary 6.2 (Residual-wire stability). *Under Q1-Q4 and the machine's reachability invariants before a fenced re-drive at f , every wire entry left after the composite carries a generation below the fence, and along any subsequent trace of arrivals and losses only, the accepted record is unchanged and exactness at f persists.*

Stale arrivals cannot be admitted by reordering or delay, and losses cannot alter A . The corollary is scoped to arrival/loss-only continuations on purpose: it says nothing about arbitrary later application work. Theorem 6 applies to a later recovery only if its conditions hold at that crash. Between recoveries, an in-flight entry is neither delivered nor lost at a completed frontier. The model keeps that ambiguity first-class rather than resolving it by fiat.

Validation controls. Two companion results locate the modeling choices. On a twin machine whose crash wipes the wire, the same schedule ends exactly-once with a duplicate-free record. Crash-survival of the wire is therefore a required assumption and a realistic choice because crashes do not empty a network's buffers or a sidecar's retry queue. And the early-arrival schedule above calibrates from the other side: arrival before the read is benign, so nothing weaker than arrival-after-read defeats reading. The model treats the fenced re-drive as one atomic transition and does not cover a crash inside it (Section 10).

7 Concurrent Recoverers and the Claim Fence

A second hazard family comes from orchestration. An orchestrator that loses a worker's heartbeat can start a second recovery while the first is still alive: two recoverers, each reading honestly, each re-driving what it read. This section is about that pair, on a machine built for it.

The concurrency machine. The machine carries $w = \langle \sigma, J, S, A, \varphi, \Gamma, hwm, fl \rangle$. Its fields are the store, a *journal* J (the record justification is measured against, whose role sharpens in Section 8), *sent* and *accepted* ledgers, a fence, a *generation table* Γ tracking each spawned generation as producing or armed with a batch, the high-water mark hwm (the newest spawned generation), and a retention *floor* fl . Delivery happens at the *fire point*: an armed generation g fires its batch, which always lands in S and lands in A exactly when the fence has not moved past it, $\varphi \leq g$. The sink delta lifts to this machine, computed against the accepted record

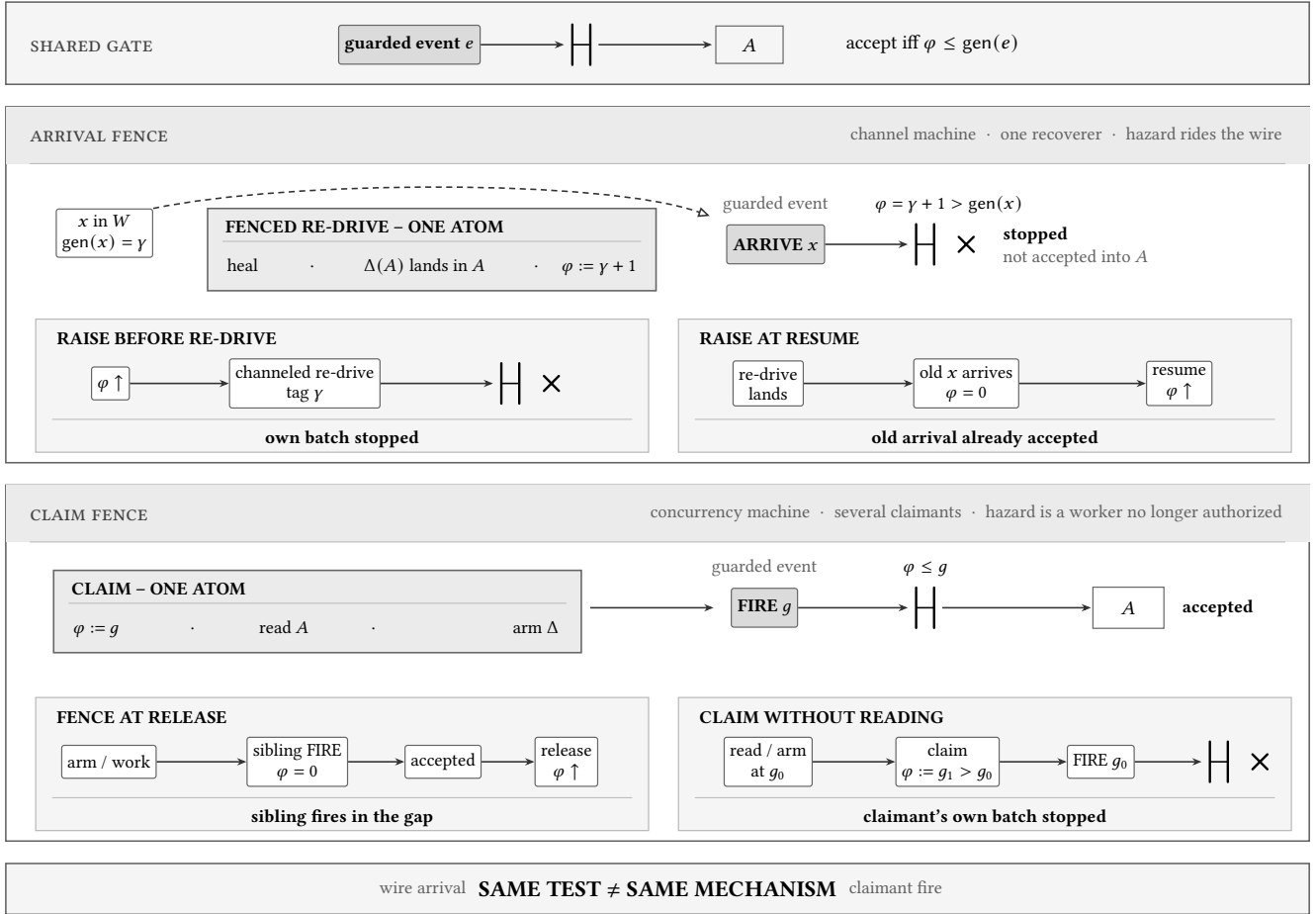


Figure 4. Arrival and claim fences use the same acceptance test for different events. The channel machine fences arrivals after an atomic re-drive. The concurrency machine fences fires after an atomic claim. The side cards show invalid reorderings. Only safety transfers between the two regimes.

over what retention has kept. There is no wire here by design. The hazards of this machine come from *workers that remain alive after losing recovery authority*, not from messages in flight, and the two hazard families do not substitute for each other. Before anything new begins, the observation bound reappears in a strengthened form. A policy reading the store, the journal's length, the fence, the whole generation table, the high-water mark, and the floor (everything short of the ledgers) is still defeated on a designed pair of this machine.

Three authorities must stay separate on this machine, because Section 8 pulls them apart. The live source history H sits inside σ . The retention-relative obligation history is $H_{\text{ret}} = \text{drop}(fl, H)$. Delta, loss, at-least-once, and exactness are evaluated against replay from H_{ret} . The journal J is the authority that justifies emissions, while A detects prematurity and duplication. Exactness on this machine is therefore *retained-view-relative* unless an additional condition identifies J , H , and H_{ret} , a relativity this section will state precisely rather than gloss.

Fence-free one-shot recovery. The negative's subject is the natural discipline (each recoverer reads, arms once, fires once) run twice in parallel without a fence.

Theorem 7 (Second-Recoverer Bound). *On the concurrency machine there is a reachable crashed state W with*

$$\varphi(W) = 0, \quad g_1 \neq g_2 \text{ both live}, \quad \neg \text{unsafe}_A(W), \\ \Delta\langle f \rangle(W) = [p].$$

The state has four properties: the fence is unset, two generations are live, the state is hazard-free, and exactly one delivery is missing at f . For every fence-free pair of one-shot recoverers, however each computes its batch with the whole state readable, some schedule in the constrained family ends in a hazard or with the delivery at f still missing. In this family, both recoverers arm, at least one fires, at least one heals, and every recoverer that heals also fires.

The defeating schedule is four or five steps, and which steps the adversary needs depends only on where the missing payload sits in the two armed batches. If both batches carry

it, let one recoverer heal and both fire. The accepted record then holds the payload twice. If neither carries it, the same steps leave the delivery incomplete. If exactly one carries it, let that recoverer die silently after arming (the family permits one silent death) while the other heals and fires its payload-free batch. The store is repaired, the run reads recovered, and the delivery is gone. Each recoverer fixed its batch before the schedule chose the cell. The family's constraints exclude trivial defeating schedules. They cannot kill both recoverers and call the loss a defeat, and a recoverer that heals cannot then vanish (every healer fires). As everywhere in this paper, the defeat is existential over schedules and says nothing about disciplines outside the fence-free one-shot class. The positive result below uses such a discipline. The sink-delta policy of Section 5 also fails on this axis. When run by two claimless recoverers, the sink delta double-fires. Two honest workers, each individually correct by Section 5's rule, are jointly wrong: each read was true when taken and stale by the time it was used.

What ordering changes. Suppose each recoverer must prepare its batch, repair the store, and then send, with no step skipped. The model permits only one store repair per crash, so these ordered schedules contain at most one completed recovery. At the constructed state, a completed sink-delta recovery is exactly-once. The duplicate in Theorem 7 therefore requires a recoverer to send without completing its store repair.

Ordering does not ensure that a batch contains the missing work. Fully ordered schedules exist for every pair of batch choices. If one batch omits the missing payload, there is an ordered schedule in which that worker completes while the other stops after preparing its batch. The delivery remains missing. The original schedule family also includes these ordered schedules.

The claim fence. An atomic *claim* by generation g requires $\varphi \leq g$, so claims never lower the fence. The claim sets the fence to g and arms the batch computed from the accepted record read in that step. The later fire is accepted only if $\varphi \leq g$, meaning it *completed at the fence*.

Lemma 7.1 (Claim-fence safety). *Every claim-disciplined trace from the machine's initial state is hazard-free.*

The safety proof requires the following restrictions in addition to the machine's usual state and history checks.

- Ordinary delivery sends only committed events that are absent from the acceptance record. It pauses while any prepared recovery batch remains authorized by the fence.
- A claim prepares a duplicate-free batch from the sink's acceptance record. The alternative cursor-recovery sequence must use a duplicate-free suffix containing only events absent from that record. It cannot start while another prepared batch remains authorized, and

its prepare, repair, and send steps must run together without intervening actions.

- Snapshot effects must match the state reconstructed from the journal prefix. These effects are used by the DBLog instance described in Section 10.

The permitted executions exclude truncating crashes, independent fence raises, and batch preparation outside a claim or the cursor-recovery sequence above. Retention, lost publishes, release, and sending an already prepared batch remain permitted.

Under these rules, the fence never exceeds the newest spawned generation. A worker can still attempt to send after a later claim removes its authority. The sink therefore checks the worker's generation when the batch is sent.

Theorem 8 (Completed-Claim Exactness). *On the concurrency machine, under*

- C1. Initialized discipline: *the trace starts at the initial state and follows the claim discipline.*
- C2. Claimed: *generation g claims at frontier f and later fires, with no intervening claim or fire of g .*
- C3. Window-fresh: *no source commit at or below f lands between the claim and the fire.*
- C4. Completed at the fence: *$\varphi \leq g$ when the fire lands, so its batch is accepted.*

the post-fire state is exactly-once at f , judged against the retained obligation history H_{ret} . Per-instance exactness additionally requires ascending coordinates. Without them the count is set-level, by payload.

Window freshness is required. A commit at or below f landing inside the claim window grows the set of required deliveries after the batch was fixed. One modeled idealization must be stated here: the fire transition executes its armed batch atomically. A recoverer crashing after a strict prefix of a multi-item batch is outside the modeled transition. Per-entry firing is future work, and Section 10 carries the absence.

Retention-relative exactness. The theorem's conclusion is graded against what retention kept, and the difference matters. The machine's retention step has no guard, so a disciplined run can raise the floor over an obligation that was committed and never delivered, dropping it from the graded history without disturbing the theorem's verdict. The mechanization includes a concrete control trace that is exactly-once relative to retained history while, judged against the journal, a committed obligation is lost. That run's accepted record is empty. The stronger, journal-graded statement is then closed under either of two conditions, and the boundary between them is itself a theorem. On the *permanent-source* fragment (journal equal to the committed history, floor zero), the two grades provably coincide, and Theorem 8 yields journal-grade exactness outright. Off that fragment, the bridge is *floor-prefix coverage*: everything retention dropped below the floor was already accepted.

Corollary 7.1 (Journal-grade exactness). *With the journal equal to the committed source history,*

$$\text{eo}^J(f, w) \longleftrightarrow \text{eo}^{\text{ret}}(f, w) \wedge \text{covered}_J(f, w),$$

where eo^J and eo^{ret} grade exactness against the journal and the retained history respectively, and $\text{covered}_J(f, w)$ means that every scoped source event at or below f that retention has excluded from replay already appears in the acceptance record. In particular, a disciplined run whose retention steps only ever drop already-accepted obligations delivers journal-grade exactness through Theorem 8.

The equivalence is the precise form of a folklore sentence (“replay what retention kept, and make sure retention only expires what was delivered”), and the guard-free counterexample above is why the second clause cannot be dropped.

Two invalid orderings. The discipline depends on this order. Two concrete counterexample traces refute the tempting variants (Figure 4, lower row). *Fencing at release* (doing the work first and setting the fence at hand-off) admits a batch from an earlier worker before the fence is raised. *Claiming without reading* (setting the fence over a batch computed before the claim) fences out the claimant’s own re-drive. Claim first, read under the claim, fire under it.

Interface between the two fences. This paper now has two fences, and no theorem merges them. The arrival fence guards a *wire*: one recoverer, arrivals from its own past. The claim fence guards a *fire point*: several recoverers, including workers that can still send after losing recovery authority. They live on different machines. What is proved between the regimes is one interface with two obligations:

- (T) *Transport delivers a sub-multiset*: what arrives is a sub-multiset of what was sent (reorder, delay, and drop are allowed, but fabrication, duplication, and alteration are not).
- (J) *Justification is monotone*: a record that justifies an emission still justifies it after any extension.

The claim discipline’s safety argument is proved once against these obligations alone, then instantiated twice: by a genuine pullback from this section’s fire point, and by a sibling abstract wire grammar that permits loss, reordering, and arrivals from earlier generations. That sibling is *not* Section 6’s channel machine and does not re-derive its theorems. The asymmetry is the interface’s shape: safety crosses it, exactness does not, and nothing else crosses at all. Note also what obligation (T) excludes: transport that *duplicates* a staged item. A generation fence rejects requests from generations below its current threshold. It does not deduplicate a same-generation copy the transport minted on its own. Deployments running over at-least-once transport need stable identity and an admission or deduplication contract in addition to anything this section proves, and Section 8 is about how long such contracts hold.

8 Evidence Lifetime

Systems often combine at-least-once delivery with deduplication [2, 4]. Theorem 9 gives a coverage identity for an ideal sink view with permanent memory. The remaining results examine bounded memory and loss of source history.

Permanent deduplication. Model the idempotent consumer as the *absorbing, permanent-memory deduplicated* view of the sink’s record, in which repeated payloads are absorbed, $\text{dedup}(t) = \text{remdups}(\text{pay}(E(t)))$, remembering every payload it has ever seen. For a list L , $\#_p L$ is the multiplicity of payload p .

Theorem 9 (Deduplicated-View Coverage Identity). *On the effect machine with the absorbing, permanent-memory view: every payload in the set of scoped replay obligations at f is represented exactly once in the deduplicated view if and only if delivery is at-least-once at f :*

$$(\forall p \in \text{replay}(H, K, f). \#_p \text{dedup}(t) = 1) \longleftrightarrow \text{alo}(f, t).$$

The view is duplicate-free by construction. The theorem establishes whether every required payload is present. It does not rule out premature effects on the underlying record or verify a consumer’s implementation of checking for a prior effect, applying the effect, and recording its completion.

The per-instance corollary additionally requires strictly increasing source coordinates. An equal-coordinate example refutes that stronger conclusion without the condition. The following results examine the assumption of permanent memory.

Bounded deduplication. Real deduplication memory is a window, and the window here is an *entry-count* horizon, measured in record positions, not seconds. A duration maps to it only through a delivery-rate argument. A legal-run counterexample shows how the guarantee fails. A windowed view satisfies the same equivalence at a reachable state, and then one more crash and one more honest re-drive push the original delivery out of the window, so the sink processes the re-driven copy as new. The windowed count reads two while permanent memory still reads one. For the constructed run the threshold is sharp between window sizes one and two, and the window-sufficiency condition that protects a run is proved sufficient and *not* necessary. The operational shape of the hazard is familiar: Stripe’s idempotency keys become eligible for removal after at least 24 hours [12], and Kafka’s producer-ID expiration setting defaults to one day, although ongoing transactions and topic retention affect when state expires [13]. An outage that outlives the deduplication horizon reproduces the incident in its quiet form: recovery is honest, the keys are gone, and the re-drive lands as new work.

Deduplicating by key. Deployed systems often match events by keys such as an LSN, offset, or idempotency

key [12]. The keyed policy compares source coordinates instead of the full payload (c, e) . It computes the same missing batch when source coordinates increase strictly and every accepted entry matches a committed source record.

Each condition has a separate counterexample. Coordinate matching can remove entries from the batch selected by full payload matching, but cannot add entries. The counterexamples show how this can suppress a required replay and leave committed work undelivered. Comparing only application contents is also insufficient. Distinct committed operations can have identical contents, and their source coordinates distinguish them. Derive the key once, at commit time, from that stable coordinate.

Compaction that keeps presence. On the sink’s own record the safe boundary is *presence*. Rewrites that preserve a record’s presence (compaction, tombstoning that keeps the key’s entry) do not affect the windowed policy, which needs no more than window-local presence. Presence-destroying retention fabricates duplicates at a reachable state. The windowed policy never loses work, so its only failure is duplication. This bounds what the policy analyzed here needs to read. That no policy could do better with bounded memory is not claimed (Section 10).

Compensation instead of deduplication. Sinks with reversing entries permit another approach: compensate, then redo. The theory analyzes three selected compensation observations (matched cancellation, signed net, and truncation), and the verdicts split. Under the matched-cancellation view, order-aware compensate-then-redo normalizes both outcomes of the designed pair. The same batch succeeds on both members because the judged observation has changed. Under the signed-net observation, the construction again forces recovery to choose the same batch for states that require different actions. Truncated views do not distinguish the pair either. Throughout, the underlying record stays append-only. Reversal operations never delete sent events, but instead append matching reversing emissions in sequence. The matched-cancellation result is proved for the abstract re-drive relation. It is not an execution of this machine’s suffix-only reconcile. Realizing it requires a compensation-capable transition, or a refinement proof for an implementation that has one.

Source truncation and journal evidence. The second shrinkage is upstream, and it is the harsher one. On the concurrency machine, retention raises the floor over a history that still exists. A *truncating crash* instead cuts the live source history itself, while the journal J remains the specification of which deliveries were required. What recovery can read afterwards is the *retained view*

$$\text{view}(w) = \langle \sigma, |J|, \varphi, \Gamma, hwm, fl, S, A \rangle,$$

everything a policy could ask for, except that the journal enters only as its *length*. Now take two runs that each commit one event, differing only in which, and let a truncating crash cut both live histories back to empty. The surviving stores are literally equal, and so is every other slot of the view: same fence, same generation table, same high-water mark, same floor, same two ledgers, and journals of the same length. The journal contents differ, hidden from the policy, and with them the sets of required deliveries. One state still has an undelivered event at f . The other requires no delivery at that frontier.

Theorem 10 (Truncation Dilemma). *On the concurrency machine there are reachable states W_1, W_2 with equal retained views over differing journals. State W_1 has an undelivered event at f according to its journal, while W_2 requires no delivery there. Every policy computed from the retained view is defeated: there are re-drives of both at f under which W_2 ’s outcome is a hazard (a fabricated delivery where none was required) or W_1 remains incomplete at f (abandonment).*

Whatever rule the policy runs, one state makes it fabricate work or the other makes it abandon work. Two follow-ups make the theorem usable. Reading the *live* source instead of the journal does not help and actively harms: the machine admits phantoms (deliveries whose justifying record is gone), and judging against the live history makes them retroactively premature, which is the machine-checked reason justification measures against the journal in the first place. And the practitioner’s version of the boundary is about *content* versus *extent*: retention that hides old content while an authoritative journal can still reconstruct it is recoverable. A crash that leaves recovery with only length-like metadata is what the theorem describes. Keep the recovery window’s journal content reconstructible, not just its size.

9 Store-Tier Characterization of Log-Derived State

CDC and the transactional outbox are the standard remedy for dual writes, and they are widely deployed: Debezium alone ships log connectors for most major databases [14]. Everything so far has treated “derive downstream state from the committed log” as sound advice whose delivery stage still needs a theory. This section proves the advice’s own half exactly, and locates the boundary between the two halves.

Store-tier characterization. At the store tier, the model is Section 2’s substrate with implementations over it. An implementation supplies an initial state, a step relation, and a projection to core states. Three properties name what the theorem asks of it:

- R1. *Crash-closed*: every reachable running state can crash at every in-interval frontier.
- R2. *Refining*: its steps project to the substrate’s.
- R3. *Running-start*: its initial state is Running.

For a materialized map h , $h \upharpoonright K$ is restriction to scoped keys. An *observable mismatch* at (c, k) means the state is Crashed c with $c \leq \text{fin}$ and the committed and downstream materializations disagree on scoped key k at c .

Theorem 11 (Faithful-Image Equivalence). *For a crash-closed, refining, running-start implementation I of the store model: I has no reachable observable scoped mismatch at a crash frontier if and only if I is running-image-faithful at every reachable Running state and every in-interval frontier f ,*

$$\text{Src}(b, D, f) \upharpoonright K = \text{Src}(b, H, f) \upharpoonright K.$$

Unlike the recovery bounds, which are constructions, this equivalence quantifies over *all* implementations of its tier: the one result in the paper that is universal over implementations, as promised in Section 2. It characterizes the equality that “derive from the log” must establish: under R1-R3, keeping the running downstream image faithful to the committed log is store-tier crash safety, in both directions. The two directions are useful in different places. Faithfulness rules out crash-frontier mismatches by a structural step: a crash changes no history, so a mismatch frozen at a crash was already present at a running predecessor, where faithfulness forbids it. That step needs R3, and R3 is necessary. An implementation that begins already crashed has no running states at all, is faithful vacuously, and is unsafe anyway. That class provides the counterexample when R3 is omitted. The converse is where R1 works: if some reachable running state held a scoped mismatch at an in-interval frontier, crash-closure supplies a crash *at that frontier*, making the mismatch observable. If R1 is weakened to crashes at only some frontiers, the argument has no crash to take.

Relay example. The two-event instance shows the biconditional at work. A downstream derived from the log in committed order has image 7 at c_1 and 9 at c_2 , matching the source at both frontiers. No crash of that run freezes a mismatch. Now let the relay acknowledge the update at c_2 but defer the write. Its delivered history still stops at the insert, so at frontier c_2 the downstream image reads 7 against the committed 9: a scoped mismatch at a running frontier, and by R1 a crash is available right there to expose it. The practitioner’s version is short: at this tier the crash adds no window of its own (what a crash can expose is exactly the gap between the committed log and the derived image at a running frontier), so derive from the log, in the log’s order, and close that gap.

A lagging relay also shows what the theorem does *not* provide. Between the acknowledgement and the deferred write, the relay is not running-image-faithful, and no theorem here says a lagging pipeline “catches up.” Liveness is outside this model. The constructive result is frontier-scoped. At every *occurrence-complete* frontier (one whose scoped obligations have all been delivered), the image is mismatch-free. CDC and outbox designs should therefore be read as machinery

for establishing complete-frontier image faithfulness at the frontiers they advertise, not as global faithfulness at every instant. An advertised frontier is a claim. The theorem says what that claim must mean.

Limit of the store tier. The designed pair of Theorem 1 agrees on the whole store, so store-tier faithfulness cannot arbitrate a single re-delivery decision. The acceptance boundary begins where this biconditional stops. The tier scales to multiple derived targets without new theory. The proof shows that fan-out diverges exactly when some single target does, and a target in the formal log-derived class (which requires image equality at every modeled frontier) is excluded from the divergence class. That is a statement about the formal class, not a blessing of every architecturally log-backed but lagging deployment. The per-target reading is the right one, and no cross-target atomicity is implied.

Derivation relocates the delivery boundary. A log-derived pipeline still has to deliver. The relay that tails H and writes the sink performs, per event, two durable effects (the delivery and its own progress record) with no shared commit between them: Section 2’s effect machine under an operational name. This relocation claim is an argument assembled from proved parts (Theorem 11 below the boundary, Theorems 1-7 above it), and we state it at that level rather than as a theorem of its own. In the incident’s terms: the relay tailing committed orders was already this stage. Derivation had removed the handler’s dual write and left the delivery loop’s own, which is where the crash found it. The replayable log lets the relay retry failed deliveries and catch up after an outage. The sink must still provide the acceptance evidence and controls recovery needs. Nor is “derive from the log” a single design point: the store tier’s replay interface has two genuinely distinct certified inhabitants, a refresh-free outbox segment and the snapshot-replay instance of Section 10, and mechanized witnesses establish their distinctness.

Two roles for an outbox. Table 1’s role split matters here. An outbox table read by the relay as its source is the committed log H . It is not thereby an acceptance record. The same physical table realizes A only if it holds sink-owned acceptance state updated atomically with acceptance. Otherwise A is a separate durable record on the sink’s side. Sentences about “the outbox pattern” that do not name the role equivocate between the two sides of the acceptance boundary: H is what Theorem 11 governs, while A is what Theorems 4 and 6 consume.

10 The Mechanization

This section explains how the recovery models relate and what the Isabelle proofs establish.

Relationship between the machines. Write ι for the embedding of effect-machine states into the concurrency machine and Π for the projection back.

Theorem 12 (Shared-Core Agreement). (i) ι embeds every effect-machine trace into the concurrency machine’s solo fragment. Projection Π maps every solo-fragment trace from a solo-invariant state back while preserving the invariant, and $\Pi \circ \iota$ is the identity. (ii) Under the alignment conditions that everything sent is accepted, the journal equals the committed history, and all payloads are log entries, the hazard verdicts premature, duplicate, and unsafe agree through Π . (iii) With the ledgers equal and the retention floor at zero, the sink-delta batch and the loss and at-least-once verdicts commute with Π .

So the small machine of Sections 3-5 really is a fragment of the large one of Sections 7-8: same verdicts, same deltas, under conditions that align the larger machine with the smaller one. The fragment excludes several behaviors. The solo grammar admits ordinary lifted steps, fused recovery, and successive generation changes, and excludes free arms, free fence raises, snapshots, lost publishes, truncating crashes, and retention. Of the shared-core laws, the three-guard fire discipline of Section 5 transports through Π . The rest were re-proved on the larger machine. Two non-transfers are structural rather than accidental: the effect machine’s absorbing recovery class does not lift (release and resume break absorption, and the successor law is open) and the wire is omitted from the concurrency model. The concurrency machine has none, so Section 6’s results neither follow from it nor imply anything on it, and the regimes touch only at Section 7’s interface. Figure 5 draws the resulting family.

Recovery cycles and the core model. Resume changes Recovered to Running and increments the generation. The histories and emissions ledger stay unchanged. The core model has no matching transition, so the proof divides executions of the cyclic machine at Resume.

Each resulting segment corresponds to an execution of the core model with relay repair added. The store-level theorem applies to sequences of label steps from the segment’s starting state, subject to the stated checks. It excludes relay repair and further Resume steps. A checked example preserves a store mismatch across Resume and repairs it after a later crash.

Prover and trust base. Every numbered result in this paper is a machine-checked theorem in Isabelle/HOL [15], checked in a mode where an unfinished proof fails the build. The trusted base consists of stock Isabelle2025-2 and, beyond the standard prelude, exactly two standard-library theories (finite multisets and subsequence orderings). There are no global axioms and no proof oracles. Theorems state their assumptions explicitly. The one named-assumption context (the acceptance interface) has its two obligations discharged

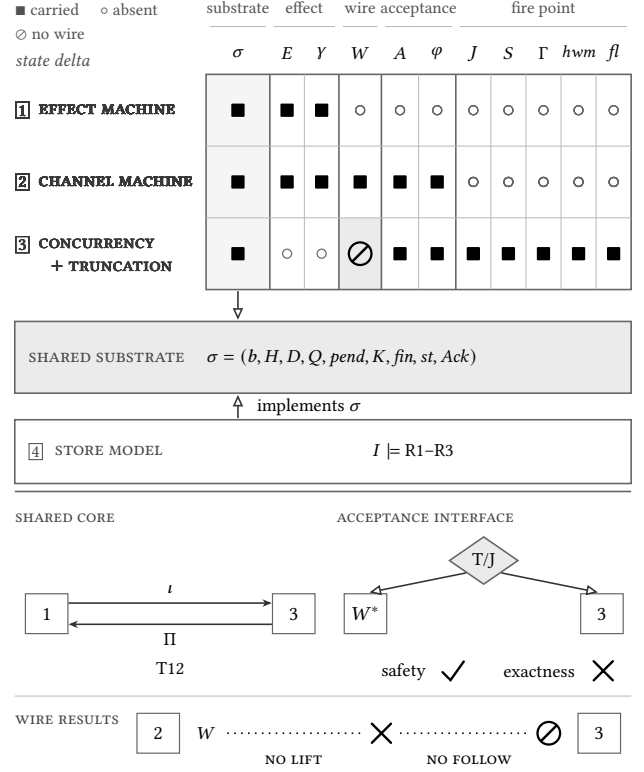


Figure 5. Fields and relations of the three recovery machines and the store tier. The checkpointed model extends the effect machine with a durable cursor and is not shown separately. Filled squares mark carried fields, and the slashed circle marks the absent wire. The lower diagrams show the shared-core embedding, the safety-only interface, and the blocked wire relation.

on both of its instances. The full development rebuilds from cold in a few minutes on a laptop.

Validation controls. The formalization includes four validation controls. Closed witnesses show that the negative and positive theorem classes are inhabited. Twin runs test the modeling choices used by the straggler result. A small-step model confirms that the observation bound does not depend on atomic recovery. A counterexample found during development required an additional condition for keyed deduplication. Scratch gate sessions also checked the main proof terms for unproved dependencies. A planted failure confirmed that this check detects an unfinished proof. These scratch sessions are not part of the released development.

Executables. Certified executables accompany the theory, chief among them an exported, theorem-correct store/hazard decider and the core store machine’s trace validator. Both are point evaluators, and their limits must be stated: a clean hazard verdict is not exactly-once (completeness at the frontier

is the second conjunct, and no hazard audit sees it), and the validator certifies label structure, not history wellformedness. In the other direction, the decider strengthens the observation bound from below. No function of the durable core alone computes even the hazard verdict.

Model boundaries. The model has six limitations.

- *A crash inside the fenced re-drive.* The composite of Theorem 6 is one atom. Nothing covers a crash between its heal, its landing, and its fence rise. An implementation must realize the atom transactionally or supply its own recovery argument.
- *Two-step publish.* A send and the sink’s readable record coincide (and a fenced recovery’s own re-sends land synchronously). A sink that acknowledges first and persists later is outside the model.
- *Per-entry claim fire.* The claim machine’s fire executes its armed batch all-or-nothing. A recoverer crashing partway through a multi-item batch is outside the modeled transition.
- *Truncation on the wire machine.* Fabricate-or-abandon is proved at the fire-point machine. The wire-tier lift is open, and no wire-tier truncation result exists here.
- *Bounded-memory converse.* Section 8 bounds what the policy analyzed here needs to read. It does not prove that every bounded-memory policy requires the same information.
- *Successor recovery-class law.* The effect machine’s recovery-cycle structure stops at the embedding of Theorem 12. Its concurrency-machine successor is open.

Scope of the model. The absences above concern mechanics. One more concerns shape. Every result in this paper concerns a single one-way pair: one authority holding a durable, ordered, per-operation record that defines the required deliveries, one endpoint whose durable acceptance defines the done work, one delivery direction between them. That shape covers the log-derived pipelines this paper grew out of, and it excludes real configurations by design: obligations running both ways, several sources feeding one sink, or one source feeding several sinks bound together rather than held each to its own pair, chains judged end to end rather than hop by hop, payloads transformed in flight rather than delivered verbatim, and pairs in which even the source keeps no committed record of what it wrote (nothing left to identify the required deliveries, so the model cannot pose the question). The shape is not an arbitrary cut: its capabilities are the same ones every result turns on. Each positive theorem states the capability it requires, and each bound is proved against a named class of policies that lacks it. The generalization this boundary map points toward is a theory that treats both endpoints as capability-graded nodes of one kind and recovers these results as the one-way, record-bearing instance.

DBLog instance. A pipeline in the style of DBLog mixes an initial snapshot or backfill with live log records, and the theory covers the mixture at the safety level. Justification is append-stable across the machine, mixed-discipline runs stay hazard-free, and the log-only fragment recovers the delivery hazards exactly as stated. Two boundaries are explicit. The snapshot modality provides safety guarantees without exactness laws. And the watermark algorithm itself is not re-proved here: the instance consumes our prior snapshot-replay development by citation [16], through a proved bridge, and that development’s own disclosed assumptions remain its own. One fact about process belongs here: at the time of writing, no independent expert has cold-read the formalization. The machine check and the controls above are the evidence on offer.

Availability. The full development is publicly archived as a permanent deposit [17], with build instructions and an index mapping every numbered statement in this paper to its mechanized counterpart.

11 Related Work

Operational guidance. The operational rule has been public for a decade, in places peer-reviewed: Kleppmann’s dual-writes argument and its book form [1, 2], the log-centric architecture literature [3], and Treat’s essay on exactly-once delivery [4]. This paper formalizes the recovery decision that remains after log-based derivation. Its theorems identify the sink acceptance evidence needed to decide what to resend and state the conditions for correct recovery.

Formal studies. Closest to our subject are bounded model checks of outbox and log-delivery designs: Masternak and Pobiega model-check an exactly-once outbox in TLA^+ [9], and LogPlayer model-checks in-order exactly-once delivery to backend shards [10]. Both predate this work. The boundary between us is method, and it matters here: a bounded check certifies the states it enumerates, while the observation bound of Sections 3-4 is a higher-order claim over *arbitrary policy functions*, which is the kind of statement finite enumeration cannot reach. Our result covers arbitrary recovery policies restricted to the modeled source-side state. We know of no earlier machine-checked proof with this scope.

Impossibility results. The classical results bound our scope rather than supply our theorems. FLP [18], in mechanized form too [19], concerns asynchronous consensus liveness. CALM [20] characterizes coordination-freeness. CAP [21] trades availability under partition. Our model concerns a different question: whether recovery can determine what an independent sink accepted. Mechanized impossibility work such as Pactole [22] provides a methodological precedent for proving limits on classes of protocols.

Atomic commitment. The classical way to remove a separable completion window is a shared commit [23], and atomic commitment across autonomous stores has its own theory and impossibility results [24, 25]. Our failure class instead has the source committed, delivery incomplete, no coordinator and no votes. Where a real shared commit spans both effects, the window this paper studies never opens. Where the sink cannot join one (the usual reason the pattern exists), these theorems apply. Nothing here bears on commit-protocol correctness.

Verified data systems. Mechanized systems work addresses several adjacent problems. VerIso [5] and mechanized serializability [26] treat isolation inside a database, with no crash-fed second store. GoJournal and Perennial [6, 7] verify one store’s crash safety, with no independently written copy. Disel [8] supports compositional verification of distributed protocols and their clients, including a two-phase-commit case study. These studies address isolation, crash safety, or protocol composition. Our focus is the recovery decision at an independently accepting sink.

Scope of deployed guarantees. The model gives deployed exactly-once machinery a vocabulary for where its guarantees stop. Kafka’s idempotent producer and transactions [11] deduplicate broker-side by producer id, epoch, and sequence (the epoch is a deployed cousin of our generation, consulted at acceptance). The producer-ID expiration setting defaults to one day. IDs do not expire during an associated ongoing transaction, and topic retention can cause earlier expiry [13]. These retention limits illustrate the finite evidence lifetime studied in Section 8. Flink’s two-phase-commit sink [27, 28] places transactional acceptance at the sink. Kafka Streams’ exactly-once guarantee covers processing from source topics to output topics and state stores [29]. That is a scope boundary, not a defect, and outside it sits the boundary this paper studies. No broker, connector, or runtime is verified here. The map says where each guarantee stops, in the model’s terms.

Prior use of fencing. Fencing by monotonic token is established engineering: Kleppmann’s fencing-token argument [30], Kafka’s producer epoch [11], and mechanized lease reasoning in Grove [31]. We claim the theorems, not the mechanism: the defeat of every channel-blind policy on a designed pair, the conditional arrival-fence result with its cost stated as a corollary, and the claim-fence guarantee, to our knowledge the first mechanized separation of that information bound from a fenced acceptance result. The corpus models no clocks and no leases, so it neither supports nor refutes lock-service designs.

Connection to DBLog. The worked instance uses the snapshot-replay model from our prior study of DBLog [16]. Section 10 uses that model as a substrate. It is not prior art for the recovery claims made here.

12 A Practitioner’s Guide to Dual Writes and CDC Recovery

This section translates the formal results into an operational guide for backend software engineers, systems architects, and reliability practitioners. Statements here are direct readings of numbered theorems or engineering guidance based on them. Throughout, the *frontier* is the position in the committed source log up to which delivery is judged, and a relay’s *generation* is its restart counter, which identifies the current run. We address two production architectures: (1) direct dual writes in application request handlers, and (2) transactional outbox and CDC pipelines with asynchronous relays.

The dual-write dilemma across one process and a CDC relay. Dual writes arise whenever an application must record related facts across two systems that do not participate in a shared transaction (like a two-phase commit). In distributed architectures, this problem manifests in two primary patterns.

Pattern 1: Direct dual writes in application handlers. An application service receives a user request (like an order checkout) and attempts to update its primary database while publishing an event to a search index, or partner API. If the database transaction commits first and the application crashes before invoking the external API, the database holds a committed record whose external side effect was never triggered. If the external API call succeeds first and the database transaction subsequently rolls back, an external side effect was released for a transaction that never existed. Network calls across boundaries are also three-valued (success, failure, or timeout): on timeout, the caller cannot determine whether the packet was lost before arrival or only the acknowledgment was lost, so retrying can duplicate side effects. Distributed transactions (XA/2PC) [23] are widely avoided in distributed architectures due to blocking latency, coordinator failure risks, and the lack of 2PC support across cloud services and external APIs.

Pattern 2: The transactional outbox and CDC relay. To eliminate direct dual writes in application code, the standard industry pattern is the transactional outbox combined with change data capture (CDC) [1, 2, 14]. The application writes its application records and an outbox event into the *same* local database transaction. A separate relay process (such as a Debezium connector or outbox worker) tails the committed database log or outbox table, delivers each event to the external sink, and records its own progress.

The relocation of the dual write. Change Data Capture relocates the dual write rather than eliminating it. Instead of occurring in the application handler, the dual write now sits in the relay delivery loop. For every event, the relay must execute two distinct durable operations under independent authorities: first delivering the event to the external sink, and second recording a local checkpoint (such as advancing an offset, watermark table, or message cursor). Because no

transaction spans the external sink and the relay local checkpoint store, a crash between delivery and checkpointing reopens the dual-write dilemma. When both source and sink are Kafka topics and consumer offsets are committed inside the producer transaction via `sendOffsetsToTransaction`, the system achieves a shared commit where the dual-write window never opens (Section 11). In heterogeneous pipelines, Theorem 11 (Section 9) shows that store-tier crash safety amounts, under its assumptions, to one condition, that the derived copy matches the committed log at every running frontier, and the delivery obligation then moves to the relay (Section 9).

Defining the acceptance boundary for API sinks and email. To design recoverable delivery, engineers must draw a clear boundary around what the delivery stage controls. Consider an application sending shipping confirmation emails through an external provider API (such as Mailgun or AWS SES), operating across two distinct regimes:

- *Durable API acceptance.* The delivery obligation is satisfied when the provider durably accepts custody of the payload. An HTTP 200 OK or 202 Accepted response containing a provider-assigned message ID evidences acceptance only if issued after durable persistence, as documented by the provider. If the provider does not document post-persistence acknowledgement, conservative recovery must treat the response as non-durable and verify delivery via the provider query API. In the formal model (Section 2), this transition marks durable acceptance: once accepted, the obligation is completed regardless of downstream transit.
- *Downstream asynchronous transit.* Everything following durable API ingestion (internal queues, MTA routing, SMTP handshakes, spam scoring, and inbox polling) lies outside the delivery stage’s authority. Transit delays must not be conflated with delivery acceptance.

This distinction applies equally to payment gateways (such as Stripe charges [12]), message brokers, and webhooks. A relay crashing while awaiting an HTTP response cannot deduce from local state whether the sink accepted the payload before the connection dropped.

Why local state cannot prove delivery. When a relay crashes and restarts, its local log and outbox records are intact, and the last durable checkpoint cursor points to event N . Whether the sink accepted event $N + 1$ before the crash or the crash occurred before transmission cannot be deduced from local state alone. If the sink accepted event $N + 1$ immediately before the crash, resending creates a duplicate. If the crash occurred before transmission, skipping drops committed work.

This ambiguity is an information limit. A local checkpoint records *the relay’s progress in persisting its own progress*

records, not the sink’s durable acceptance. In a post-crash state where delivery succeeded but the checkpoint write failed, the source database, write-ahead log, outbox table, and cursor look bit-for-bit identical to a state where delivery never occurred. Any source-measured recovery policy must choose the same action in both states, duplicating delivery in one or silently omitting it in the other (Theorems 1-2, Section 3).

Committing a checkpoint after every single message reduces the replay backlog during recovery, but does not eliminate the crash window. It merely cycles the window once per event. Theorem 3 (Section 4) strengthens the bound to this exact setting: for a deterministic protocol that checkpoints after every send, crash timing alone constructs two reachable post-crash states that agree on the local cursor and disagree on sink acceptance.

Classifying sinks by recovery capability. Recovery depends on two distinct capabilities exposed by the sink, namely what the sink lets recovery *read* (visibility) and what the sink enforces at *admission* (control). Practical sinks fall into three archetypes (summarized in Table 2).

Archetype 1: Blind or write-only sinks. When the sink API provides no query endpoint to inspect past acceptances and does not support idempotency keys or fencing, recovery cannot determine what was delivered. No recovery policy that consults only source-side state can achieve exactly-once delivery at the recovery frontier (Theorem 1). The system must explicitly select its failure preference by either resending on ambiguity (accepting duplicates) or skipping on ambiguity (accepting omissions). Client-side retries and backoff do not remove this trade-off [4].

Archetype 2: Queryable logs and stores. When writing to a sink whose accepted history can be read back (such as a Kafka topic or secondary database table), recovery can query the sink to inspect which events are present. By computing the difference between the committed source log and the sink’s accepted records, recovery resends only missing events (the Sink-Reading Escape of Theorem 4, Section 5). Three operational conditions govern this approach:

- *Coordinate monotonicity (Condition P4).* Source coordinates must be strictly increasing per event in commit order (such as a database WAL LSN or an outbox event coordinate assigned at commit). Auto-increment sequence numbers generated at insert time can commit out of order under concurrent transactions, which breaks the commit-order assumption the frontier relies on and can leave committed events below a frontier unrecovered.
- *The credential constraint.* Production services provisioned with write-only permissions (such as Kafka produce rights without consume rights) silently degrade an Archetype 2 sink into an Archetype 1 blind sink.

Table 2. Sink capability archetypes and corresponding recovery contracts.

Sink archetype	Examples	Read and admission capability	Recovery contract
Blind or write-only	Basic HTTP webhooks, write-only queues, legacy email APIs	Cannot query past accepted events, with no admission deduplication or fencing tokens.	No source-measured policy can guarantee exactly-once delivery at the recovery frontier. The system must explicitly choose between duplicate retries or omission under ambiguity.
Queryable log or store	Kafka topic (with read ACLs), replicated datastore table, queryable event store	Recovery can inspect sink history to determine which source events are missing, but the sink does not natively deduplicate.	Compute difference between committed source log and sink records, and resend only missing events (Theorem 4). Requires strictly ascending coordinates (P4) and generation fencing against stragglers and concurrent recoverers.
Deduplicating or fenced	Stripe API (idempotency keys), Kafka transactional producer (transactional.id + epoch), cloud queues with FIFO deduplication	Sink admission gate deduplicates by client-supplied key or rejects stale producer epochs.	Deduplicate by stable event key, or compute missing events from authoritative acceptance records and use fenced recovery. Retain deduplication state throughout the recovery horizon.

- *Acceptance history versus current state.* In a key-value store, reading current state (such as status = "shipped") distinguishes some crash states, but fails if intermediate updates were overwritten. Recovery requires a record of per-operation acceptance. Compaction that preserves key presence is safe (Section 8), but presence-destroying retention (such as hard-deleting records or purging tombstones) destroys queryable evidence. Sinks read from replicas must also enforce strong read consistency to avoid replica lag hiding recent writes.

Archetype 3: Deduplicating and fenced sinks. Stripe deduplicates requests using client-supplied idempotency keys [12]. Kafka’s stable transactional.id supports transaction recovery and fences old producers [11]. A relay can still resend an event already committed to Kafka. It must also commit its progress with the output or use the sink’s authoritative acceptance record and the fencing rules below to send only missing events. With permanent deduplication memory and strictly increasing coordinates, at-least-once delivery gives exactly-once coverage (Theorem 9, Section 8). In practice, finite memory requires a retention window long enough to cover replay. Derive idempotency keys from stable source coordinates and keep them unchanged across retries. Payload hashes alone can merge distinct operations with identical contents and suppress required delivery.

Controlling in-flight stragglers and concurrent recoverers. Even with queryable or deduplicating sinks, two distinct distributed systems hazards can invalidate recovery.

Hazard 1: In-flight network stragglers. Suppose Relay 1 sends an event to a queryable sink, but crashes while the packet remains buffered in a retrying network proxy or transport queue. A recovery worker restarts, queries the sink, observes that event $N + 1$ is missing, and resends event $N + 1$,

which the sink accepts. Moments later, the delayed packet from Relay 1 arrives at the sink. If the sink lacks arrival fencing, it accepts the delayed request, causing a duplicate. Theorem 5 (Section 6) proves that no channel-blind policy reading only durable state can detect in-flight stragglers. The proved remedy is the atomic fenced re-drive (Theorem 6), where the sink atomically accepts the recovery delta and advances its generation threshold to one above the crashed run’s generation, rejecting older requests at admission. In this model, recovery uses the crashed generation. Fencing first blocks its requests, while fencing afterward leaves a window for stragglers. Recovery with a new generation falls outside this argument (Section 6). *The cost of arrival fencing.* While arrival fencing guarantees exactness at the recovery frontier, Corollary 6.1 proves that it can reject delayed requests that carried work needed at subsequent frontiers, leaving that subsequent work as a separate unfulfilled obligation.

Hazard 2: Concurrent recoverers. In containerized environments, a recovery worker losing its heartbeat can cause an orchestrator to launch a replacement worker. If both workers run concurrently, both query the sink, observe event $N + 1$ missing, and fire re-drive batches. Distributed lock leases do not prevent this hazard [30]. If a worker pauses and its lease expires, a replacement worker claims leadership, but the original worker can wake and fire its prepared network call. Theorem 7 (Section 7) proves this Second-Recoverer Bound, establishing that without an admission fence, concurrent recoverers can duplicate deliveries or lose work. The proved mitigation is atomic claim fencing (Lemma 7.1 and Theorem 8), under which a recoverer atomically sets the sink fence to its own generation and arms its batch from the sink record read under that claim. The fire step succeeds at the sink only if the fence has not been advanced past the worker’s generation. Theorem 8 requires window freshness (Condition C3, requiring that no source commit

occurs within the recovery range between claim and fire) and validates generation authority at the sink admission gate (Condition C4).

Fencing internal versus external SaaS sinks. Kafka enforces epoch fencing natively at admission via transactional producer epochs. Relational datastores implement the fence as a conditional generation check inside the same transaction that applies the re-drive batch. External SaaS APIs lacking epoch endpoints (such as Stripe) cannot enforce claim fencing directly. Source outbox leases reduce redundant calls under normal operation, but cannot stop a resumed worker from firing. For external APIs, admission deduplication via deterministic idempotency keys is the sole barrier absorbing stragglers. As Section 7 notes, no theorem merges arrival and claim fencing; they address distinct failure models on different machines.

Retention horizons across deduplication caches and source logs. Deduplication state cannot be retained indefinitely in real systems.

- Stripe’s documentation specifies that idempotency keys become eligible for removal after at least 24 hours [12].
- Apache Kafka’s producer ID expiration (`producer.id.expiration.ms`) defaults to 1 day [13], though topic log retention can cause earlier expiry.
- Cloud FIFO queues configure sliding deduplication windows (such as a 5-minute window from message acceptance).
- Upstream database outbox tables undergo WAL or bin-log purging.

Replay after retention expiry. If an upstream database undergoes a prolonged outage or network partition and recovery replays historical events after the sink’s deduplication cache has expired, the sink treats the replayed requests as novel transactions, triggering a wave of duplicate side effects.

Sizing and retention guidance.

- **Sink deduplication horizon.** The theoretical model in Section 8 measures deduplication memory in entry-count horizons. In practice, converting duration to entry capacity requires bounding the maximum arrival rate. The retention horizon must exceed the longest anticipated recovery window: $T_{\text{dedup}} > T_{\text{max_outage}} + T_{\text{operator_delay}} + T_{\text{replay_catchup}} + \text{margin}$. If deduplication evidence expires before replay finishes, the sink admits replayed events as novel (Section 8). When fixed provider windows (such as a 5-minute FIFO deduplication window) cannot be widened, replays exceeding the window cannot rely on the sink for deduplication. If the queue cannot report past acceptances, recovery after expiry is blind, as in Archetype 1. An auxiliary record helps only if it is authoritative and updated

atomically with queue acceptance. Separate sender-side bookkeeping leaves the crash window open.

- **Source history retention.** To maintain journal-grade exactness, upstream outbox records must be retained until the downstream sink has durably confirmed their acceptance (Corollary 7.1). Prematurely truncating source history removes the content needed to identify which operations require delivery, leaving only coordinate bounds or metadata. With the historical content lost, recovery is forced to choose between phantom fabrication and abandonment (Theorem 10).

Architecture checklist, auditing, and recovery tests.

To translate these boundaries into practice, teams can apply the following checklist, monitoring strategy, and recovery tests.

Architecture checklist.

- **Monotonic event coordinates.** Assign stable event IDs at source commit time strictly increasing in commit order (such as a database WAL LSN or an outbox event coordinate assigned at commit). Auto-increment sequence numbers generated at insert can commit out of order under concurrent transactions. Never generate fresh random UUIDs during retries.
- **Documented sink contracts.** Explicitly classify each sink as Blind, Queryable, or Deduplicating. For Blind sinks, record the failure preference (duplicates versus omissions).
- **Arrival fencing for in-flight stragglers.** For internal sinks, ensure recovery atomically raises the generation fence upon re-drive, preventing delayed in-flight requests from prior runs from landing.
- **Claim fencing for concurrent recoverers.** Ensure replacement recovery workers claim leadership and validate generation tokens at the sink admission gate before firing batches.
- **Verified read permissions.** Ensure relay service accounts possess explicit read/consume permissions where queryable recovery is assumed.
- **Dead-letter queue trade-off.** Routing unprocessable payloads to a Dead-Letter Queue (DLQ) prevents head-of-line blocking, but accepts a frontier omission. The reconciliation monitor must record these coordinates as acknowledged omissions (avoiding persistent alerts), and the DLQ write itself represents an auxiliary dual-write boundary.
- **Outbox storage safeguards.** Because outbox records must be retained until downstream confirmation (Corollary 7.1), extended sink outages cause outbox tables to accumulate rows. Implement threshold alerts on row count and WAL utilization to prevent storage exhaustion.

Monitoring and reconciliation auditing across frontiers. Delivery failures are asymmetrical in production. Duplicate

side effects are loud (prompting complaints or billing alerts), whereas omissions are silent (a skipped event can leave dashboards green). To detect silent omissions on Queryable or Deduplicating sinks, implement an independent *reconciliation auditor*, a background job querying the source database for committed event coordinates and verifying their presence in the sink’s acceptance log between the last verified position and the latest completed frontier. Alert whenever a committed obligation remains unaccepted past the SLA.

Recovery tests. Dual-write and CDC pipelines should be verified in staging against failure scenarios tailored to sink archetypes:

- *The checkpoint crash (all archetypes).* Terminate the relay immediately after the sink ingests a request but before the local checkpoint commits. For Archetype 1 (Blind), verify the system produces the configured compromise (duplicate retry or skip). For Archetypes 2 and 3, verify that restart does not create duplicate effects.
- *The in-flight straggler (Archetype 2).* Inject artificial latency on a delivery request, trigger a timeout and recovery restart, let recovery complete, and then release the delayed original request. Verify that the sink rejects the straggler at its generation fence.
- *The split-brain worker (Archetypes 2 and 3).* Pause an active recovery worker, allow a failover worker to claim leadership and complete recovery, then resume the original worker. Verify that the superseded worker’s writes are rejected by the claim fence.
- *The expired deduplication replay (Archetype 3).* Fast-forward cache expiry or flush the sink’s deduplication state, then trigger a manual replay of historical events. Verify that operational safeguards reject blind replays across expired retention windows.
- *Crash inside fenced recovery.* Inject a crash during recovery re-drive. Because the model abstracts fenced re-drive as an atomic transition (Section 10), implementations execute distinct steps (reading accepted history, submitting re-drive, and raising the fence). Verify that a crash before fence advancement leaves the system recoverable without duplicate side effects.

13 Conclusion

In this paper, we developed a machine-checked theory of recovery when a process delivers an effect to one system and records its progress in another. We proved that the durable state of the crashed side cannot decide whether a delivery should be repeated. Two reachable states can share identical source databases, logs, control states, restart counters, and checkpoints while differing in what the sink accepted. Any policy consulting only this shared state must duplicate an effect in one execution or omit committed work in the

other. We also proved this result for a deterministic deliver-then-checkpoint protocol in which crash timing is the only nondeterminism.

The positive results identify the controls required for exact recovery. An authoritative, complete, and current record of sink acceptance allows recovery to send only the committed operations the sink has not yet accepted. This works only if nothing changes the record between the read and the re-send. For requests still in flight from the crashed run, recovery uses an arrival fence. For overlapping recoverers, an atomic claim ties the read to the worker that made it, and the sink accepts the batch only while that claim is still the latest. These mechanisms solve distinct hazards on different machines, with conditions and costs that form part of the guarantee.

The guarantee also depends on evidence retention. With permanent deduplication, every required payload appears once in the deduplicated view exactly when each has been delivered at least once. A finite deduplication window can admit replays as new after identity expiry, and truncating source history removes the information needed to identify undelivered operations. Practical recovery must therefore retain sink acceptance evidence and source history for the full recovery horizon.

Transactional outboxes and change data capture remain the right tools for deriving downstream state from a commit log. At every recovery source-log position, downstream state must match the state obtained by replaying the committed log to that position. This agreement is necessary and sufficient for store-level crash safety under the model’s assumptions. The relay created by that design has its own delivery boundary. Reviews of that boundary should identify the sink’s durable acceptance event, active requests or recoverers, and retention horizons on both sides. A source checkpoint tracks progress, but cannot replace the sink’s acceptance record.

AI assistance. Generative AI assistants Claude Fable 5 (Anthropic) and ChatGPT 5.6 Sol (OpenAI) supported drafting, revision, and Isabelle/HOL formalization. The author directed the research, verified all proofs and text, and takes full responsibility for this work. The assistants are tools, not authors.

References

- [1] Martin Kleppmann. Using logs to build a solid data infrastructure (or: Why dual writes are a bad idea). Confluent Blog, <https://www.confluent.io/blog/using-logs-to-build-a-solid-data-infrastructure-or-why-dual-writes-are-a-bad-idea/>, 2015.
- [2] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O’Reilly Media, 2017.
- [3] Martin Kleppmann, Alastair R. Beresford, and Boerge Svingen. Online event processing. *Communications of the ACM*, 62(5):43–49, 2019. doi: 10.1145/3312527.

- [4] Tyler Treat. You cannot have exactly-once delivery. Brave New Geek blog, <https://bravenewgeek.com/you-cannot-have-exactly-once-delivery/>, March 2015.
- [5] Shabnam Ghasemirad, Si Liu, Christoph Sprenger, Luca Multazzu, and David Basin. VerIso: Verifiable isolation guarantees for data-base transactions, 2025. URL <https://arxiv.org/abs/2503.06284>. arXiv:2503.06284.
- [6] Tej Chajed, Joseph Tassarotti, Mark Theng, Ralf Jung, M. Frans Kaashoek, and Nickolai Zeldovich. GoJournal: A verified, concurrent, crash-safe journaling system. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2021)*, pages 423–439, 2021. URL <https://www.usenix.org/conference/osdi21/presentation/chajed>.
- [7] Tej Chajed, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. Verifying concurrent, crash-safe systems with Perennial. In *Proceedings of the 27th Symposium on Operating Systems Principles (SOSP 2019)*, pages 243–258, 2019. doi: 10.1145/3341301.3359632.
- [8] Ilya Sergey, James R. Wilcox, and Zachary Tatlock. Programming and proving with distributed protocols. *Proceedings of the ACM on Programming Languages*, 2(POL):28:1–28:30, 2018. doi: 10.1145/3158116.
- [9] Tomasz Masternak and Szymon Pobiega. Model checking exactly-once. Exactly Once blog, TLA⁺ models: <https://github.com/exactly-once/model-checking>, Post: <https://exactly-once.github.io/posts/model-checking-exactly-once/>, July 2020.
- [10] Mohammad Roohitavaf, Kun Ren, Gene Zhang, and Sami Ben-Romdhane. LogPlayer: Fault-tolerant exactly-once delivery using gRPC asynchronous streaming, 2019. URL <https://arxiv.org/abs/1911.11286>. arXiv:1911.11286.
- [11] Apache Kafka community. KIP-98: Exactly once delivery and transactional messaging. Apache Kafka Improvement Proposals. <https://cwiki.apache.org/confluence/display/KAFKA/KIP-98+-Exactly+Once+Delivery+and+Transactional+Messaging>, 2017.
- [12] Stripe. Idempotent requests. Stripe API reference. https://docs.stripe.com/api/idempotent_requests, 2026. Accessed 9 September 2026.
- [13] Apache Kafka documentation. Broker configuration producer.id.expiration.ms. https://kafka.apache.org/43/configuration/broker-configs/#brokerconfigs_producer.id.expiration.ms, 2026. Default 86400000 ms. KIP-854: Separate configuration for producer ID expiry. Accessed 9 September 2026.
- [14] The Debezium project. Debezium: Change data capture for a variety of databases. <https://debezium.io>, 2026. Accessed 9 September 2026.
- [15] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. Isabelle/HOL: A Proof Assistant for Higher-Order Logic, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002. doi: 10.1007/3-540-45949-9.
- [16] Andreas Andreakis. A theoretical study of DBLog: Certified virtual cuts for a snapshot-equivalent replay of live databases, 2026. URL <https://arxiv.org/abs/2605.31475>. arXiv:2605.31475. Formal development archived on Zenodo, concept DOI 10.5281/zenodo.20389696 (version 2.1 at DOI 10.5281/zenodo.21732790).
- [17] Andreas Andreakis. Isabelle/HOL formal development for “Machine-Checked Dual-Write Recovery from a Commit Log”, 2026. URL <https://doi.org/10.5281/zenodo.22700396>. Version 1.0.1. Archival deposit, Zenodo.
- [18] Michael J. Fischer, Nancy A. Lynch, and Michael S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985. doi: 10.1145/3149.214121.
- [19] Benjamin Bisping, Paul-David Brodmann, Tim Jungnickel, Christina Rickmann, Henning Seidler, Anke Stüber, Arno Wilhelm-Weidner, Kirstin Peters, and Uwe Nestmann. A constructive proof for FLP. Archive of Formal Proofs, <https://www.isa-afp.org/entries/FLP.html>, 2016. Companion: Mechanical verification of a constructive proof for FLP, *ITP 2016*, LNCS 9807, pages 107–122. https://doi.org/10.1007/978-3-319-43144-4_7.
- [20] Joseph M. Hellerstein and Peter Alvaro. Keeping CALM: When distributed consistency is easy. *Communications of the ACM*, 63(9): 72–81, 2020. doi: 10.1145/3369736.
- [21] Seth Gilbert and Nancy Lynch. Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. *ACM SIGACT News*, 33(2):51–59, 2002. doi: 10.1145/564585.564601.
- [22] Cédric Auger, Zohir Bouzid, Pierre Courtieu, Sébastien Tixeuil, and Xavier Urbain. Certified impossibility results for Byzantine-tolerant mobile robots. In *Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, volume 8255 of *Lecture Notes in Computer Science*, pages 178–190. Springer, 2013. doi: 10.1007/978-3-319-03089-0_13.
- [23] Jim Gray. Notes on data base operating systems. In *Operating Systems: An Advanced Course*, volume 60 of *Lecture Notes in Computer Science*, pages 393–481. Springer, 1978. doi: 10.1007/3-540-08755-9_9.
- [24] James G. Mullen, Ahmed K. Elmagarmid, Won Kim, and Jamshid Sharif-Askary. On the impossibility of atomic commitment in multidatabase systems. In *Proceedings of the Second International Conference on Systems Integration (ICSI)*, pages 625–634. IEEE, 1992.
- [25] Rachid Guerraoui. Revisiting the relationship between non-blocking atomic commitment and consensus. In *Distributed Algorithms, 9th International Workshop (WDAG ’95)*, volume 972 of *Lecture Notes in Computer Science*, pages 87–100. Springer, 1995. doi: 10.1007/BFb0022140.
- [26] Dmitriy Traytel. Formalization of (conflict)-serializability and strict two-phase locking. Archive of Formal Proofs, <https://www.isa-afp.org/entries/Serializable.html>, 2025.
- [27] Apache Flink documentation. TwoPhaseCommitSinkFunction. Flink 1.20 Javadoc. Deprecated in favour of the newer Sink API. <https://nightlies.apache.org/flink/flink-docs-release-1.20/api/java/org/apache/flink/streaming/api/functions/sink/TwoPhaseCommitSinkFunction.html>, 2026. Accessed 9 September 2026.
- [28] Piotr Nowojski and Mike Winters. An overview of end-to-end exactly-once processing in Apache Flink (with Apache Kafka, too!). Apache Flink blog, February 2018. URL <https://flink.apache.org/2018/02/28/an-overview-of-end-to-end-exactly-once-processing-in-apache-flink-with-apache-kafka-too/>.
- [29] Apache Kafka documentation. Kafka Streams core concepts: Processing guarantees. <https://kafka.apache.org/43/streams/core-concepts/>, 2026. processing.guarantee. KIP-129: Streams Exactly-Once Semantics, 2017. Accessed 9 September 2026.
- [30] Martin Kleppmann. How to do distributed locking. <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>, February 2016.
- [31] Upamanyu Sharma, Ralf Jung, Joseph Tassarotti, M. Frans Kaashoek, and Nickolai Zeldovich. Grove: A separation-logic library for verifying distributed systems. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP 2023)*, pages 113–129, 2023. doi: 10.1145/3600006.3613172.